010123138

Computer Programming



Check **Attendance..**



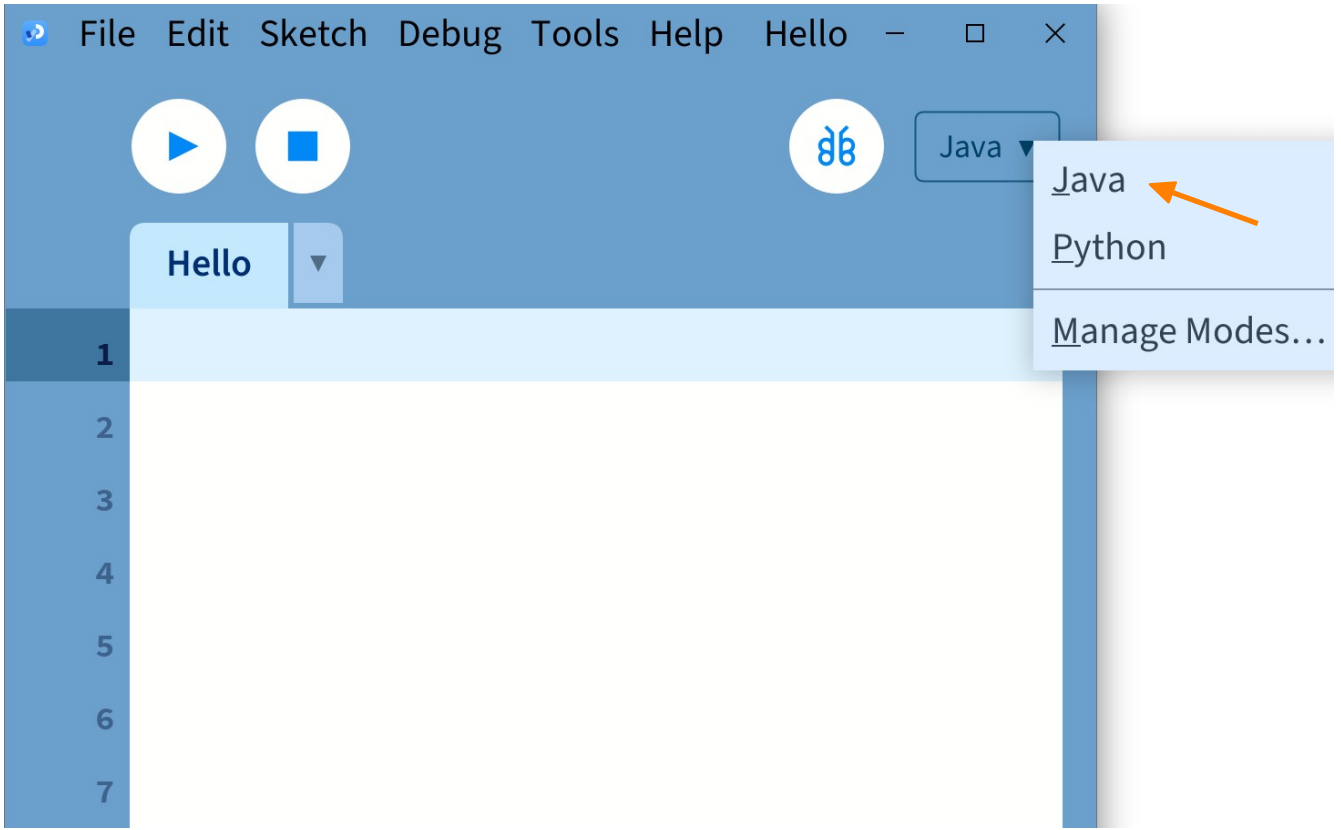
Need a Volunteer



Hello Java



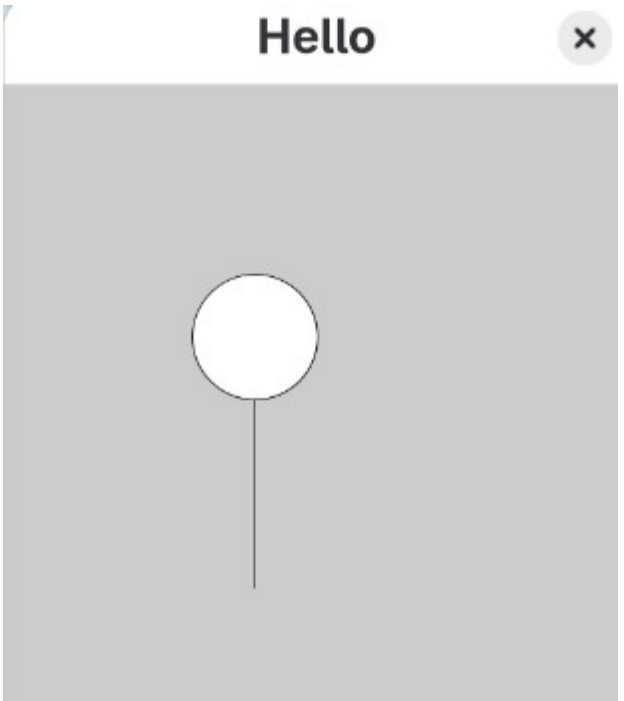
Start with Java-mode Processing



setup() in Python-mode



```
def setup():
    size(500, 500)
    ellipse(200,200,100,100)
    line(200,250,200,400)
```



setup() in Java-mode



```
void setup() {
    size(500, 500);
    ellipse(200,200,100,100);
    line(200,250,200,400);
}

// “void” == “def” ?   (please wait)
```

Same function?

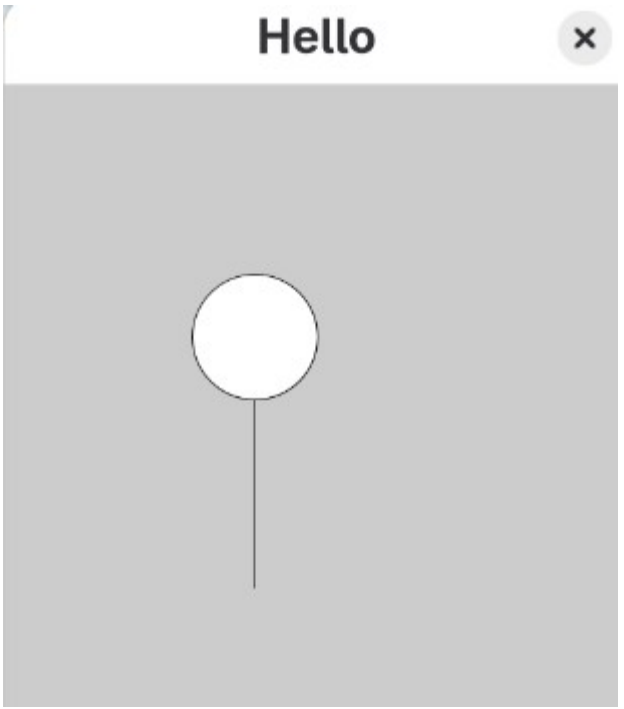


```
void setup() {
    size(500, 500);
    ellipse(200,200,100,100);
    line(200,250,200,400);
}
// Java comment (use two slashes)
// The name of the function is setup()
```

Same result?



```
void setup() {
    size(500, 500);
    ellipse(200,200,100,100);
    line(200,250,200,400);
}
// setup() works as before
```



Semicolon ;



```
void setup() {
    size(500, 500);  // must put semicolon at the end
    ellipse(200,200,100,100);
    line(200,250,200,400);
}
```

Curly Brackets { }



```
void setup() {  // use { at the begin of block
    size(500, 500);
    ellipse(200,200,100,100);
    line(200,250,200,400);
}               // use } at the end of block
```

Block of code



```
void setup() {  // use { at the begin of block
    size(500, 500);
    ellipse(200,200,100,100);
    line(200,250,200,400);
}               // use } at the end of block

// should also indent 4 spaces as in Python
```

Java do not care about space/indent



```
void setup() {size(500,
500);ellipse(200,200,100,100);line(200,250,200,400);

}



// Give the same result, but should not do it
```

. . .

User-defined function



```
void draw_balloon(){
    ellipse(200,200,100,100);
    line(200,250,200,400);
}

// “void” == “def” ?  (please wait a little more)
```

Semicolon ;



```
void draw_balloon(){
    ellipse(200,200,100,100);
    line(200,250,200,400);
}

// same as earlier
```

Curly Brackets { }



```
void draw_balloon(){
    ellipse(200,200,100,100);
    line(200,250,200,400);
}

// same as earlier
```

Block of code



```
void draw_balloon(){
    ellipse(200,200,100,100);
    line(200,250,200,400);
}

// same as earlier
```

Function Call



```
void setup(){
    size(500,500);
    draw_balloon();
}


// Just call its name (as in Python)
// and follow it with a semicolon
```

Function Parameter



```
void draw_balloon(... x, ... y){
    // specify the name of parameter
    ...

}
```

Function Parameter



```
void draw_balloon(int x, int y){
    // specify the type
    ...

}
```

Function Parameter



```
void draw_balloon(int x, int y){
    // specify the type
    // int == integer
    ...
}
```

Function Parameter



```
void draw_balloon(int x, int y){

    ...


}
```

Function Parameter



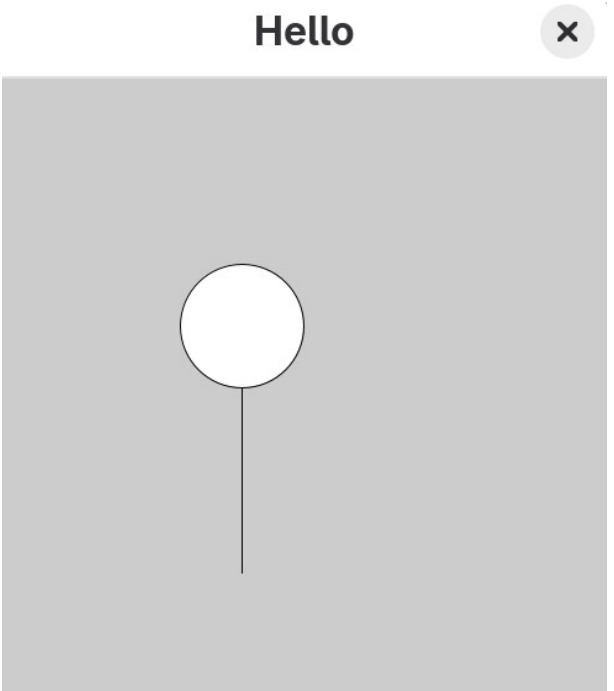
```
void draw_balloon(int x, int y){
    ellipse(x, y, 100, 100);
    line(x, y+50, x, y+200);
}
// as before
```

Function Call



```
void setup(){
    size(500,500);
    draw_balloon( 200 , 200 );
}


// send arguments
// same concept
```



Call it twice



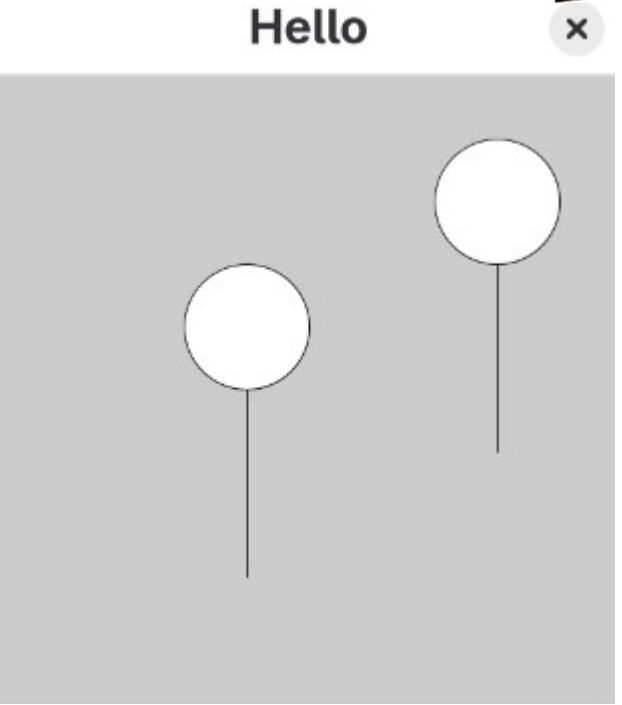
```
void setup(){
    size(500,500);
    draw_balloon( 200 , 200 );
    draw_balloon( 400 , 300 );
}
 // send different arguments
 // same concept
```

Call it twice



```
void setup(){
    size(500,500);
    draw_balloon( 200 , 200 );
    draw_balloon( 400 , 300 );
}


 // as expected
```



Animation



What are needed?

Animation



```
global variable

setup()

draw()
```

Global Variable



```
int x=100;

// must specify its type

// similar to parameter
```

Global Variable



```
int x=100;
 // must specify its type
 // similar to parameter


// In native Java language
// global variables are not allowed
```

OOP



```
// In native Java language

// global variables are not allowed


Java tries to enforce an OOP-style structure

   everything must be inside a class
```

draw()



```
int x = 100;

void setup(){
    size(500,500);
}

void draw(){
    draw_balloon(x, 200);
    x = x + 10;
}
```

draw()



```
void draw(){
    draw_balloon(x, 200);
    x = x + 10;
}
// same concept as in Python-mode
```

...

if-statement



```
void draw(){
    draw_balloon(x, 200);
    x = x + 10;
    if (x > 500){
        x = 100;
    }
}
// same concept as in Python-mode
```

Block of code



```
void draw(){
    draw_balloon(x, 200);
    x = x + 10;
    if (x > 500){
        x = 100;
    }
}
// See also: if-else and nested if statements
```

Built-in Global variable



```
void draw(){
    draw_balloon(x, 200);
    x = x + 10;
    if (x > width){
        x = 100;
    }
}
// same concept as in Python-mode
```

Local Variable



```
void draw(){
    int x = 90;
    draw_balloon(x, 80);
}
// must specify its type
// same syntax as global variable
```

Built-in Function



```
void draw(){
    int x = random(100,400);
    draw_balloon(x, 80);
}
// Error?
// random() return floating-point number
// Type mismatch: cannot convert from float to int
```

Type Casting



```
void draw(){
    int x = (int) random(100,400);
    draw_balloon(x, 80);
}
// (int) will try to cast to int
```

Type Casting



```
void draw(){
    int x = (int) "70" ;
    draw_balloon(x, 80);
}
// (int) will try to cast to int
// Error: Cannot cast from String to int
```

String Parsing



```
void draw(){
    int x = Integer.parseInt( "70" );
    draw_balloon(x, 80);
}
// Call a method (parseInt)
// in class Integer
```

. . .

User defined Function
(with return value)



```
... cal_middle(... x1, ... x2) {


   // To calculate the middle of x1 and x2
   // and return the result


   // type of parameters?


}
```

User defined Function
(with return value)



```
... cal_middle(int x1, int x2) {


   // To calculate the middle of x1 and x2
   // and return the result
     return (x1 + x2)/2


   // type of return value?
}
```

Integer Division



```
print( 5/2 );


// In Java (and many programming languages)
// the result is 2
// when both operands are integer


print( 5/2.0 );  // when one operand is float?
```

User defined Function
(with return value)



```
float cal_middle(int x1, int x2) {


// To calculate the middle of x1 and x2
// and return the result
  return (x1 + x2)/2.0 // return floating point value


// type of return value
}
```

User defined Function
(with return value)



```
float cal_middle(int x1, int x2) {

// To calculate the middle of x1 and x2
// and return the result
  return (x1 + x2)/2.0  // return floating point value


// type of return value
}
```

User defined Function
(with return value)



```
... cal_distance(... x1, ... y1, ... x2, ... y2) {


   // To calculate the distance between 2 points

   // and return the result  (from w3)


   // exercise

}
```

User defined Function
(with no return value)



```
void draw_balloon(int x, int y){
    ellipse(x, y, 100, 100);
    line(x, y+50, x, y+200);
}


// void == empty
// the function does not return a value
```

Function return type



```
void setup(){

    ...

}


// void == empty

// the function does not return a value
```

Function return type



```
... draw(){

    ...

}


// void == empty

// the function does not return a value
```

Function return type



```
void draw(){

    ...

}


// void == empty

// the function does not return a value
```

Function return type



```
float cal_middle(int x1, int x2) {

  // To calculate the middle of x1 and x2
  // and return the result
    return (x1 + x2)/2.0  // return floating point value


  // type of return value
}
```

. . .

Array of variables



```
a = [ 5, 9, 2 ] # Python-mode
```

Array of variables



```
a = [ 5, 9, 2 ] # Python-mode

int[] a = ...

// specify its type, [] mean ...
```

Array of variables



```
a = [ 5, 9, 2 ] # Python-mode

int[] a = ...

// specify its type, [] mean array
```

Array of variables



```
a = [ 5, 9, 2 ] # Python-mode

int[] a = ...

// int[] mean array of integers
```

Array of variables



```
a = [ 5, 9, 2 ] # Python-mode

int[] a = { 5, 9, 2 };

// initialise its values, use { }
```

While Loop



```
int[] a = { 5, 9, 2 };
int i = 0;
while (i < ...){

    // print all elelments

}
```

While Loop



```
int[] a = { 5, 9, 2 };
int i = 0;
while (i < a.length){ // length is an attribute of a

    // print all elelments

}
```

While Loop



```
int[] a = { 5, 9, 2 };
int i = 0;
while (i < a.length){
    print( a[i] );                    // no new line?
    i = i + 1;
    // similar to Python?
}
```

592

println()



```
int[] a = { 5, 9, 2 };
int i = 0;
while (i < a.length){
    println( a[i] );            // will add a new line
    i = i + 1;
    // it is similar to Python print()
}
```

```
5
9
2
```

For Loop



```
int[] a = { 5, 9, 2 };

for(int i = 0; i < a.length; i = i + 1){
    println( a[i] );
}

// same flowchart as while loop
```

For Loop



```
int[] a = { 5, 9, 2 };

for(int i = 0; i < a.length; i = i + 1){
    println( a[i] );
}

// put important parts together at the top
// similar in many programming languages, but not Python
```

For-each Loop



```
int[] a = { 5, 9, 2 };

for(int x : a){
    println( x );
}

// similar to for loop in Python?
```

Array of variables



```
int[] a = { 5, 9, 2 }; // initialise its values, use { }

int[] b = ...          // not initialise its values
```

Array of variables



```
int[] b = new int[5]; // array of size/length = 5


// create a new array
```

Array of variables



```
int[] b = new int[5]; // array of size = 5

// the size is fixed, cannot be changed

// learn how to solve this in the next subject
```

Array of variables



```
int[] b = new int[5]; // array of size = 5
b[0] =  2;
b[1] =  7;
b[3] =  8;
// initialise later
```

Array of variables



```
int[] b = new int[5];
b[0] =  2;
b[1] =  7;
b[3] =  8;
// initialise later
// how about b[2] and b[4]?
```

Array of variables



```
int[] b = new int[5];
b[0] =  2;
b[1] =  7;
b[3] =  8;
// initialise later
// how about b[2] and b[4]?
// Java will initialise them to zero
// Not all languages do that, better do it yourself
```

Array of variables



```
int[] b = new int[5];
b[0] =  2;
b[1] =  7;
b[3] =  8;

// exercise: random the values
```

...

Class



A class is a blueprint/template for creating objects

An Object is an item/entity/คน/สัตว์/สิ่งของ

Specify what you are interested in

- Attributes (Data/state)
- Methods (Behaviour/action/calculation/function)

Create Class



```
class Balloon { // specify class name



}
```

Add Attributes



```
class Balloon { // specify class name
// specify its attributes
    int x;
    int y;
    int r;

}
```

Add Constructor



```
class Balloon {
    ...
    Balloon(int x, int y, int r){  // specify constructor
        this.x = x;
        this.y = y;
        this.r = r;
    }
// similar concept to Python, this == self
```

Constructor



```
class Balloon {
    ...
    Balloon(int x, int y, int r){  // specify constructor
        this.x = x;
        this.y = y;
        this.r = r;
    }
// the name of constructor is the same as the class name
```

Constructor



```
class Balloon {
    ...
    Balloon(int x, int y, int r){   // specify constructor
        this.x = x;
        this.y = y;
        this.r = r;
    }
// Same name as the class and no return type
```

Add Method



```
class Balloon {
    ...
    void show(){
        line(this.x, this.y, this.x, this.y + 100);
        ellipse(this.x, this.y, this.r, this.r);
    }
}
// similar concept to Python, this == self
```

Create an Object



```
void setup(){
    size(500,500);
    Balloon t = new Balloon(200, 200, 70);
    // create a new Balloon object
}
```

Call its Method



```
void setup(){
    size(500,500);
    Balloon t = new Balloon(200, 200, 70);
    t.show();
}
```

Add another Method



```
void move(){

    // exercise

}
```

Animation



```
global variable

setup()

draw()
```

Global Variable



```
// int x=100;
// must specify its type

Balloon t;
// Declare t as a variable
// that can refer to a Balloon object
// but not create an object yet
```

setup()



```
Balloon t; // declare t as a variable...


void setup(){
    size(500,500);
    t = new Balloon(200, 200, 70); // create object
}
// create new object of class Balloon
```

draw()



```
void draw(){
    background(255);
    t.show();
    t.move();
}

// similar to Python
```

Array of objects



```
//Balloon t; // declare t as a variable...

Balloon[] b;

// Declare b as a variable

// that can refer to an array of Balloon objects

// but not create an object yet
```

setup()



```
Balloon[] b; // declare b as a variable...


void setup(){
    size(500,500);
    b = new Balloon[ 5 ]
    ...
}
```

setup()



```
void setup(){
    size(500,500);
    b = new Balloon[ 5 ]
    for (int i = 0; i < b.length; i=i+1) {
        int x = (int) random(50, 450);
        int y = (int) random(50, 500);
        int r = (int) random(30, 80);
        b[i] = new Balloon(x, y, r);
    }
}
```

draw()



```
void draw(){
    background(255);
    for (int i = 0; i < b.length; i=i+1) {
        b[i].show();
        b[i].move();
    }
}
```

. . .

To try Native Java



- Install JDK (Java SDK)
- Find and follow Java tutorials
- Compile java source code (to create byte code)

  e.g. javac Hello.java
- Run java byte code (on JVM)

  e.g. java Hello

...



010123138

Computer Programming



Check **Attendance..**



Data Science



Data Science



Process of acquiring knowledge through data

Data Science



Process of acquiring knowledge through data

Interdisciplinary field that uses data to

- understand patterns
- answer questions
- make predictions
- support decision-making

Data Science



Process of acquiring knowledge through data

Data --> Information --> Knowledge

Data



Data (The Raw Facts)

Raw, unorganized, and unprocessed facts, numbers

Factory machine logs

e.g. "Machine A: Break down date : 12/3/25"

Information



Information (Processed Data)

Data that has been cleaned, processed, organized, and structured
to give it meaning and make it easier to understand and use.

By summarize logs, get monthly report showing

Machine A break down 3 times
Machine B break down 1 times
Machine C ...

Knowledge



Knowledge (Actionable Understanding)

Information that has been understood and combined to provide insights, patterns, or rules that can guide human decisions and actions

Machine A break down 3 times, is there a problem with it? May check its past history to determine whether it should be replaced.

Knowledge



Example: Hospital Patient Monitoring System

Data (Raw Facts):

Patient's heart rate, oxygen, and temperature log

Information (Processed Data):

Patient's heart rate and temperature have been increasing for 4 hours

Knowledge (Understanding):

Doctors recognize that these changes may indicate a problem, such as infection or internal bleeding, so they can check the patient and take action early.

Knowledge



Example: Customer Reviews

Data (Raw Facts):

Customer star ratings: 5, 4, 2, 1, 5, 3, 2, 1

Information (Processed Data):

Most low ratings mention slow delivery

Knowledge (Understanding):

Delivery speed is a key factor affecting customer satisfaction, so improving delivery time may improve customer experience

Data Science Workflow



Identifying a problem

Data Collection

Data Processing

Exploratory Data Analysis (EDA)

Data Modeling

Communication

Data Science Workflow



Identifying a problem

define the business problem, formulate hypotheses, and decide what metrics matter

AI can help identify anomalies or hidden opportunities in historical data that humans might miss

Data Science Workflow



Identifying a problem

Data Collection

Write program, querying databases, etc.

AI tools automate data extraction
For example, Computer Vision models can extract text from scanned documents or images
Natural Language Processing (NLP) can scrape, parse, and clean unstructured text from the web

Data Science Workflow



Identifying a problem

Data Collection

Data Processing

handling missing values, removing duplicates, correct values, detect outlier

AI may assist

Data Science Workflow



Identifying a problem

Data Collection

Data Processing

Exploratory Data Analysis (EDA)

Data Modeling

Communication

Data Science Workflow



Exploratory Data Analysis (EDA)

Examine the data to discover patterns, relationships, trends, and unusual observations

Use statistics and visualizations such as charts, graphs, and correlation analysis

Data Science Workflow



Data Modeling

Build statistical or machine-learning models to answer the problem or make predictions

Select an appropriate algorithm, train the model, and evaluate its performance.

Data Science Workflow



Communication

Present the findings and results to stakeholders in a clear and understandable way
(use reports, dashboards, visualizations)

Explain the key insights and how they can support decision-making

...

Data Visualization



Raw numbers vs. visual patterns

Data Visualization



Raw numbers

x,y

10.0,8.04

8.0,6.95

13.0,7.58

9.0,8.81

11.0,8.33

14.0,9.96

6.0,7.24

4.0,4.26

12.0,10.84

7.0,4.82

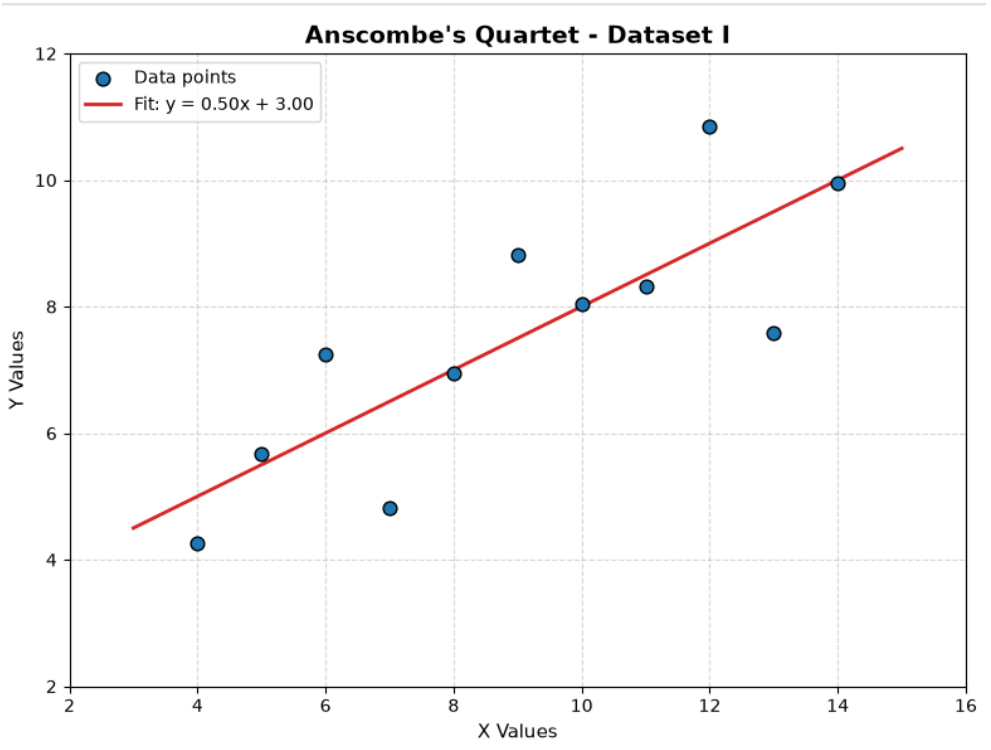
5.0,5.68

Data Visualization



Raw numbers vs. visual patterns

```
x,y
10.0,8.04
8.0,6.95
13.0,7.58
9.0,8.81
11.0,8.33
14.0,9.96
6.0,7.24
4.0,4.26
12.0,10.84
7.0,4.82
5.0,5.68
```



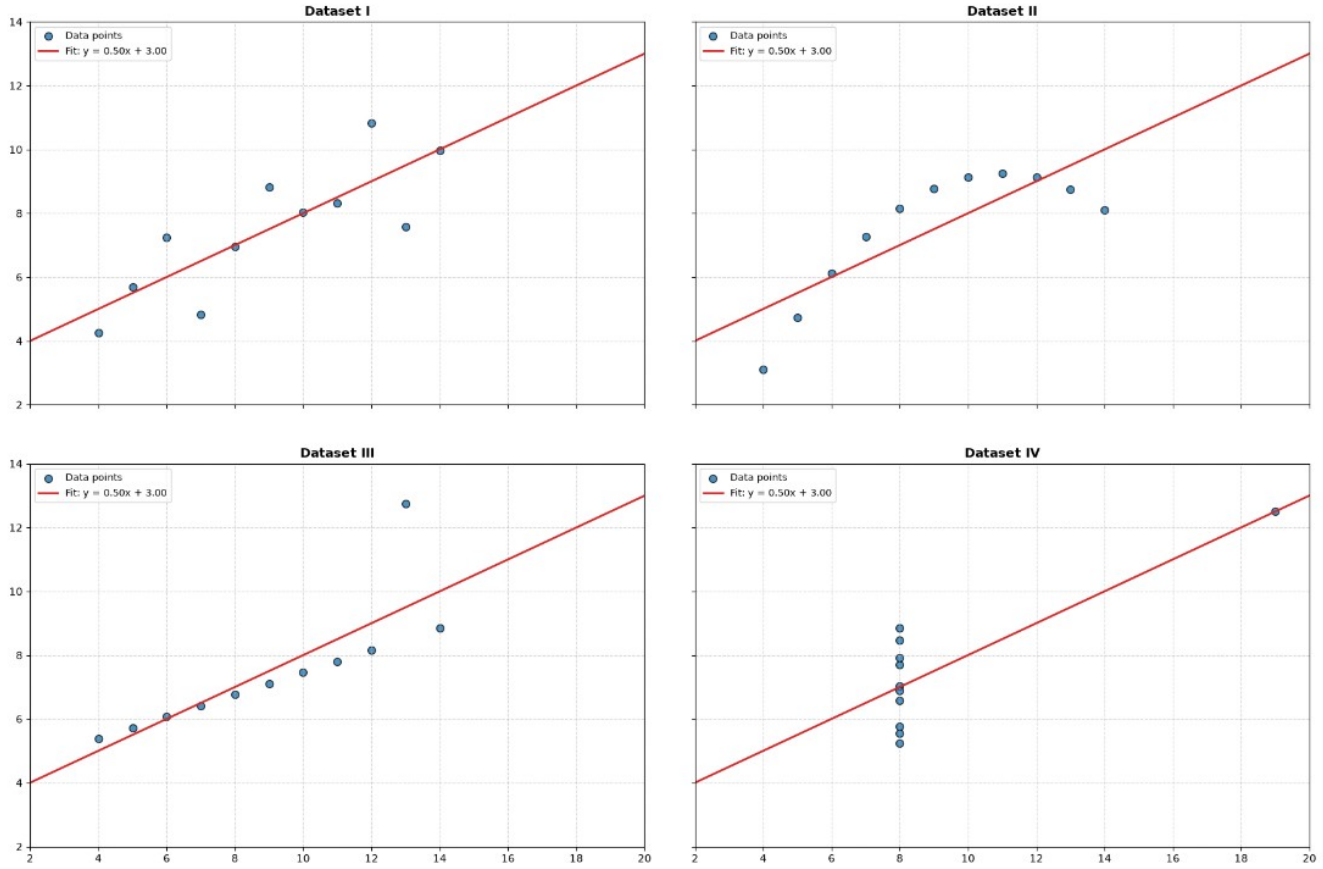
Similar Statistics



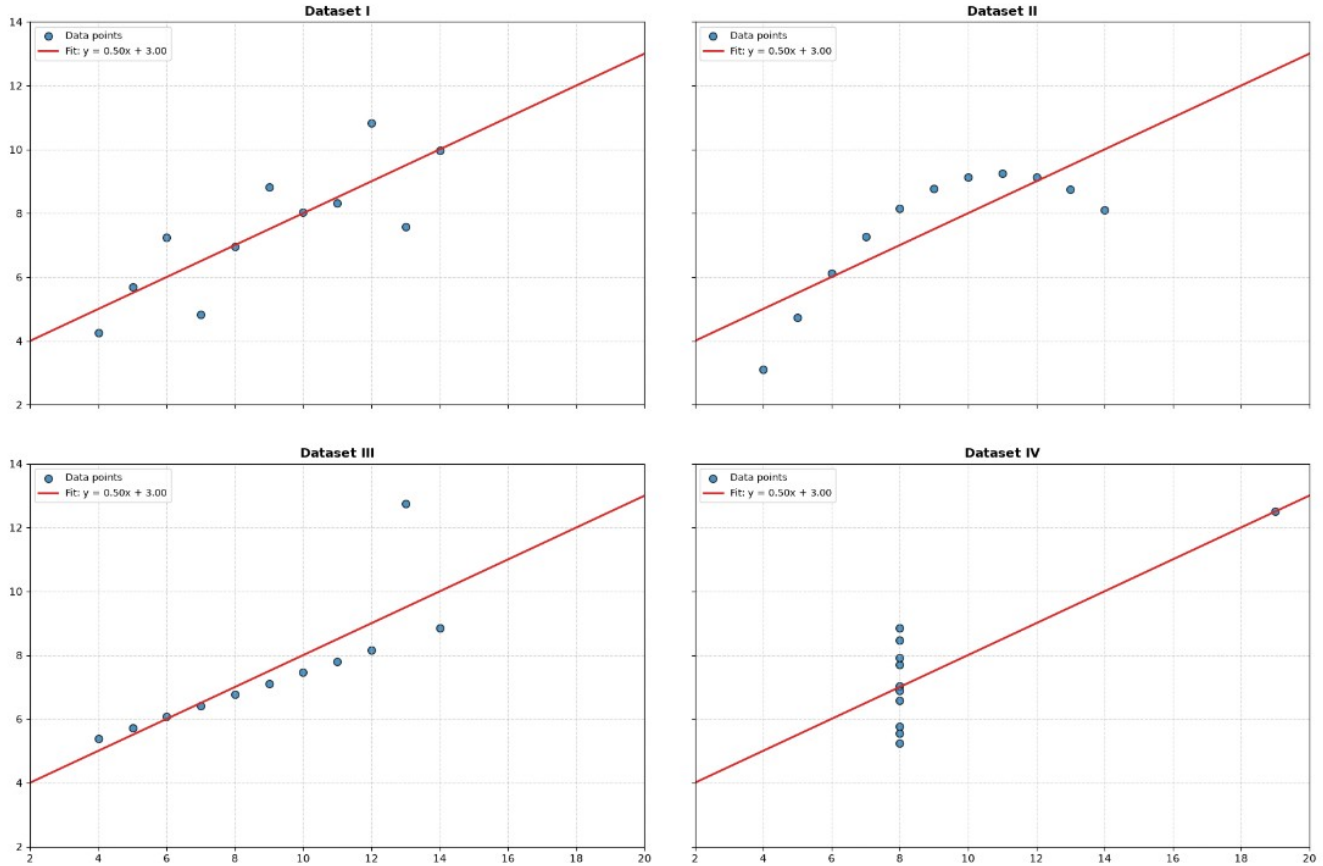
	Dataset I	Dataset II	Dataset III	Dataset IV
Mean_x	9.000000	9.000000	9.000000	9.000000
Variance_x	11.000000	11.000000	11.000000	11.000000
Mean_y	7.500909	7.500909	7.500000	7.500909
Variance_y	4.127269	4.127629	4.122620	4.123249
Correlation	0.816421	0.816237	0.816287	0.816521
Linear Regression slope	0.500091	0.500000	0.499727	0.499909
Linear Regression intercept	3.000091	3.000909	3.002455	3.001727

Anscombe’s Quartet

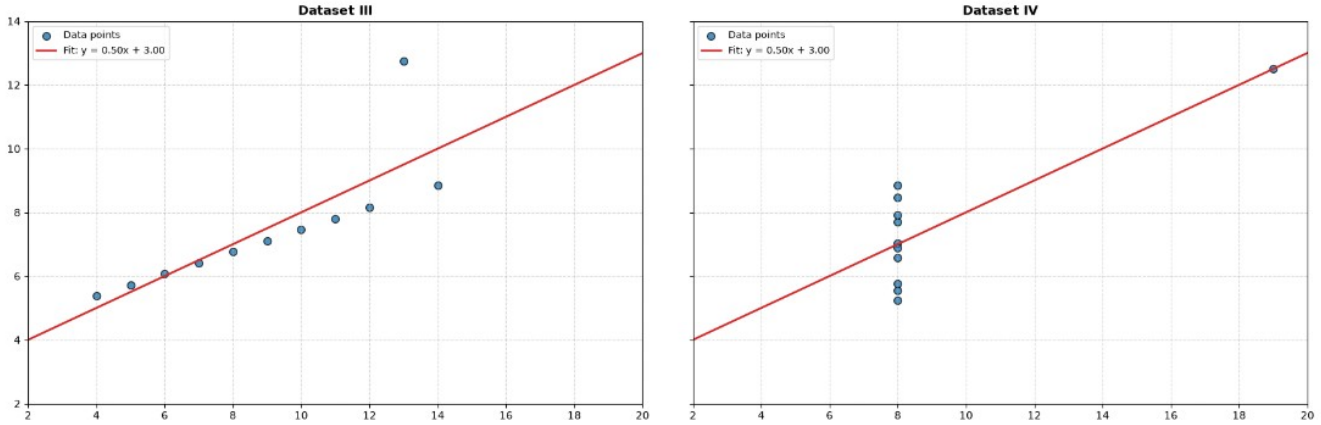
Different Patterns



Outlier


Dataset I
Data points
Fit: y = 0.50x + 3.00
Dataset II
Data points
Fit: y = 0.50x + 3.00
Dataset III
Data points
Fit: y = 0.50x + 3.00
Dataset IV
Data points
Fit: y = 0.50x + 3.00

Outlier



Outlier - data value that is significantly different from the other values in a dataset

Should be processed carefully

not automatically remove an outlier, investigate why it is different

...

Matplotlib



A Python library for creating charts

Matplotlib



A Python library for creating charts

- line chart
- bar chart
- pie chart

Line chart



```
import matplotlib.pyplot as plt


x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]


plt.plot(x, y)
plt.title("Simple Line Plot")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

Line chart



```
import matplotlib.pyplot as plt


x = [1, 2, 3, 4, 5]      # x values
y = [2, 3, 5, 4, 10]     # y values
...
```

Line chart



```
...
plt.plot(x, y, linestyle='--', marker='o')
# plot using x and y
# set line style and marker
```

Line chart



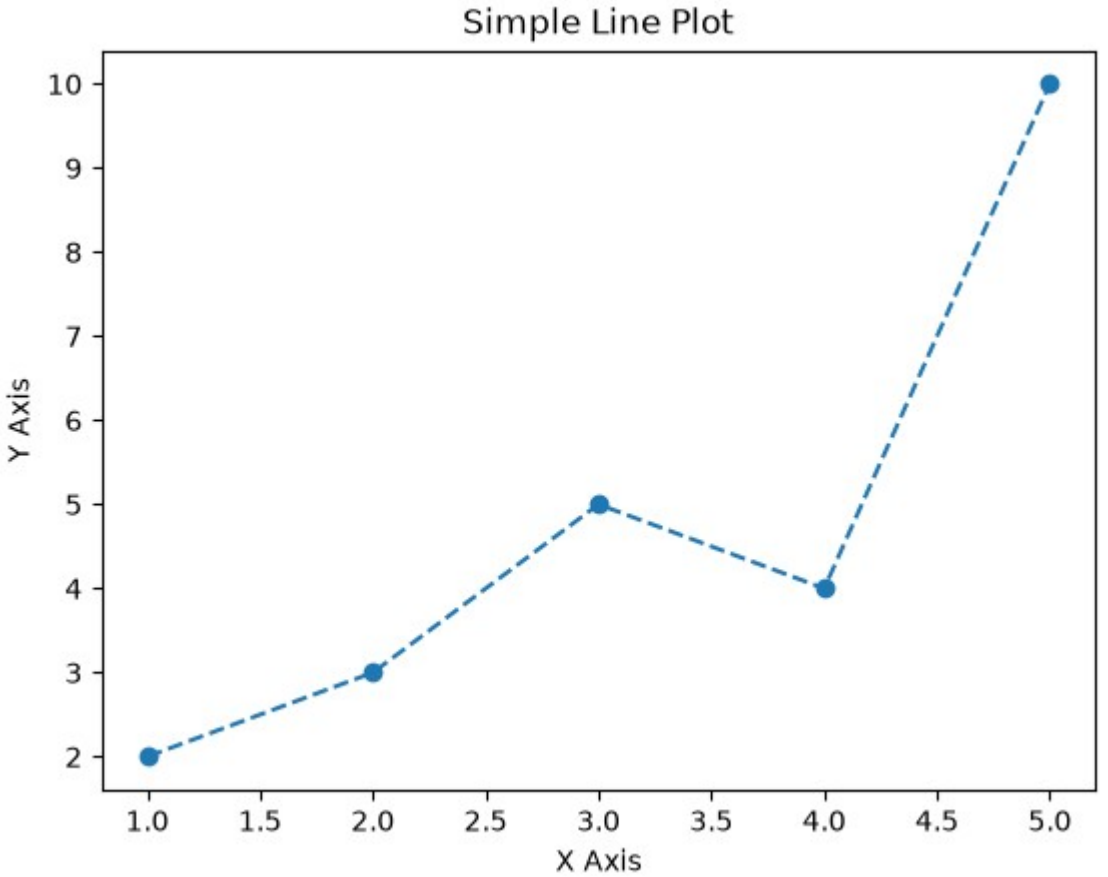
```
...
plt.plot(x, y, linestyle='--', marker='o')
plt.title("Simple Line Plot")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
# set title, labels
```

Line chart

```
import matplotlib


x = [1, 2, 3, 4,
y = [2, 3, 5, 4,


plt.plot(x, y, li
plt.title("Simple
plt.xlabel("X Axi
plt.ylabel("Y Axi
plt.show()
```



Bar chart



```
import matplotlib.pyplot as plt


products = ['Product A', 'Product B', 'Product C']
revenue = [45000, 78000, 32000]
...
```

Bar chart



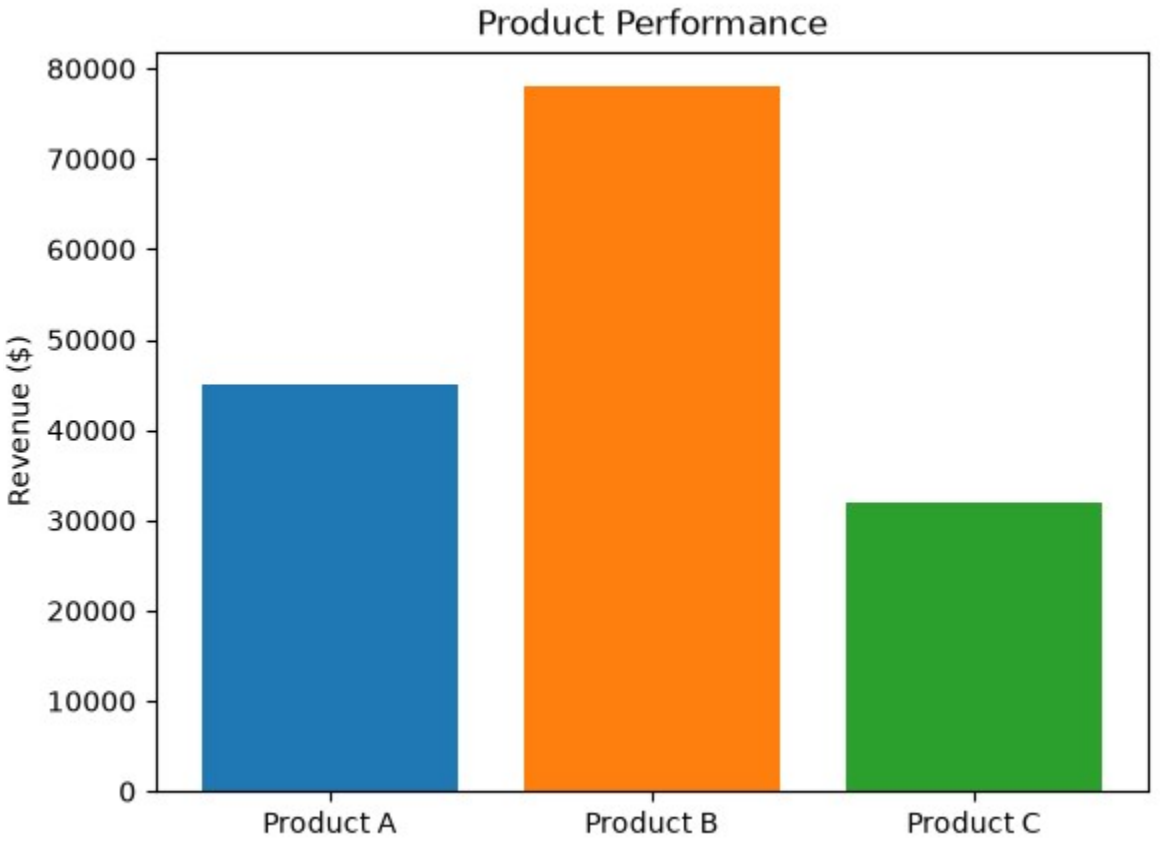
```
...
plt.bar( products, revenue, \
        color=['#1f77b4', '#ff7f0e', '#2ca02c'])
...
```

Bar chart



```
...
plt.bar( products, revenue, \
        color=['#1f77b4', '#ff7f0e', '#2ca02c'])
plt.ylabel('Revenue ($)')
plt.title('Product Performance')
plt.show()
```

Bar chart


Product Performance
Revenue ($)
80000
70000
60000
50000
40000
30000
20000
10000
0
Product A
Product B
Product C

Pie chart



```
import matplotlib.pyplot as plt


hobbies = ['Gaming', 'Reading', 'Exercise', \
           'Gardening']

hours = [10, 6, 5, 3]

...
```

Pie chart



```
...

plt.pie( hours, labels=hobbies)


...
```

Pie chart



```
...
plt.pie( hours, labels=hobbies)


plt.title('Weekly Time Spent on Hobbies')
plt.show()
```

Pie chart


Weekly Time Spent on Hobbies
Gaming
Reading
Gardening
Exercise

Simple to use library



To create chart

```
...  plt.plot(x, y, linestyle='--', marker='o')

...  plt.bar( products, revenue, ...

...  plt.pie( hours, labels=hobbies)
```

...

Guide Lines



Color & Contrast: Use readable color palettes; avoid clashing high-saturation colors

Chart Junk: Minimize unnecessary gridlines, 3D effects, or redundant labels

Make it clear to see the result

To try Python Libraries



NumPy - Numerical computations

Pandas - Data exploration, manipulation, and preparation

Scikit-learn - Machine learning model building, training, and evaluation to make predictions

...

A2 w14 update



1. Please add w14-update.pdf to your repository

In w14-update.pdf, explain what you have done in each commit made after w13-update.

For each commit, briefly describe:

- What you changed or implemented

- Any important fixes or improvements

- Any other relevant information about the commit

A2 w14 update



2. Submit a text file containing the output of command:

```
git log --oneline
```

For example:

```
a1b2c3d Fix input validation

d4e5f6g Add XYZ functionality

...
```

A2 w14 update



2. Submit a text file containing the output of command:

```
git log --oneline
```

Name the file using your student ID:

s<student-id>-w14-log.txt

For example:

s69-010126-5577-2-w14-log.txt

A2 w14 update



Before submitting, make sure:

- w14-update.pdf has been added to your repo.
- w14-update.pdf explains every commit after w13
- s<student-id>-w14-log.txt contains the output of git log --oneline.
- You have used your own student ID in the filename.
- Your repository is up to date.

...

010123138

Computer Programming



Check **Attendance..**



Review



Need a Volunteer



Class



A class is a blueprint/template for creating objects

An Object is an item/entity/คน/สัตว์/สิ่งของ

Specify what you are interested in

- Attributes (Data/state)
- Methods (Behaviour/action/calculation/function)

User-defined



User-defined Variable

User-defined Function

User-defined Type (data type/Class)

Object-Oriented Programming (OOP)



OOP is a programming approach that organizes programs around objects

"Procedural programming" organizes a program as a sequence of procedures or functions

(what we did at the beginning)

4 Pillars of OOP



The four main concepts of Object-Oriented Programming

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

Inheritance



Inheritance – Creating a new class based on an existing class

* Subclass will inherit (from Superclass)
    - Attributes
    - Methods

Subclass



```
class EsBalloon( Balloon ):

    # EsBalloon is a sublcass of Balloon
    # for expanding and shinking balloon

    ...
```

Subclass



```
class EsBalloon( Balloon ):

    def __init__(self, x, y, r):

        # define the constuctor

        ...
```

Subclass



```
class EsBalloon( Balloon ):

    def __init__(self, x, y, r):

        super().__init__(x, y, r)

        # call the constuctor of the superclass

        ...
```

Subclass



```
class EsBalloon( Balloon ):

    def __init__(self, x, y, r):

        super().__init__(x, y, r)

        self.change = 1

        # add a new attribute
```

Add a method



```
def expand_shrink(self):
    self.r = self.r + self.change

    if self.r >= 60:
        self.change = -1
    elif self.r <= 30:
        self.change = 1
```

Setup



```
def setup():
    size(1000, 1000)
    x_coords = [100, 200, 400, 500, 600]
    y_coords = [900, 850, 600, 700, 950]

      i = 0
      while i < len(x_coords):
          balloons.append( EsBalloon(x_coords[i], y_coords[i], 50) )
          i = i + 1
```

Draw



```
def draw():
    background(255)
    i = 0
    while i < len(balloons):
        balloons[i].show()
        balloons[i].move()
        balloons[i].expand_shrink()
        i = i + 1
```

Draw



```
def draw():
    background(255)
    i = 0
    while i < len(balloons):
        balloons[i].show()
        balloons[i].move()
        # balloons[i].expand_shrink() # include it in move()?
        i = i + 1
```

Method Override



Inheritance – Creating a new class based on an existing class

* Subclass will inherit (from Superclass)
  - Attributes
  - Methods

Can **override** a method of the superclass

  - move() in EsBalloon may include expand_shrink()

Method Override



```
class EsBalloon(Balloon):

    ...

    def move(self):
        # override move() in the superclass
        # so it will not do the same thing

        ...
```

Method Override



```
class EsBalloon(Balloon):

    ...

    def move(self):   # override move() in the superclass

        super().move()   # call the superclass method

        self.expand_shrink()   # call its own method
```

Polymorphism



Polymorphism – Many shapes/forms

Polymorphism



Polymorphism – Many shapes/forms

- Method overriding

  A method with the same name
  can have different behaviour

move() in EsBalloon can be different from its superclass

Polymorphism



Polymorphism – Many shapes/forms

- Method overriding
- Subtype

  EsBalloon is a Balloon?

  Person can be a student?

  have the same set of methods?

. . .



User Interface



A User Interface (UI) is the system through which a human interacts with a computer

Command-Line Interface

Early user interfaces were primarily text-based Command-Line Interface (CLI)



users interacted with computers by typing in commands

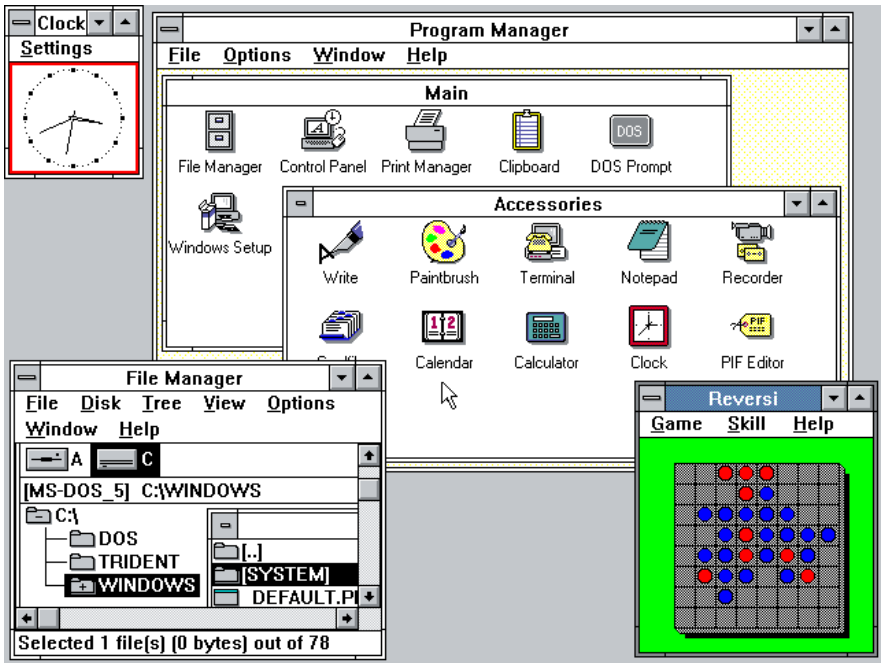
e.g. dir, cd, md

ls, mkdir

https://en.wikipedia.org/wiki/VT100

Graphical User Interface

Graphical User Interface (GUI)



users interacted with computers by pointing, clicking

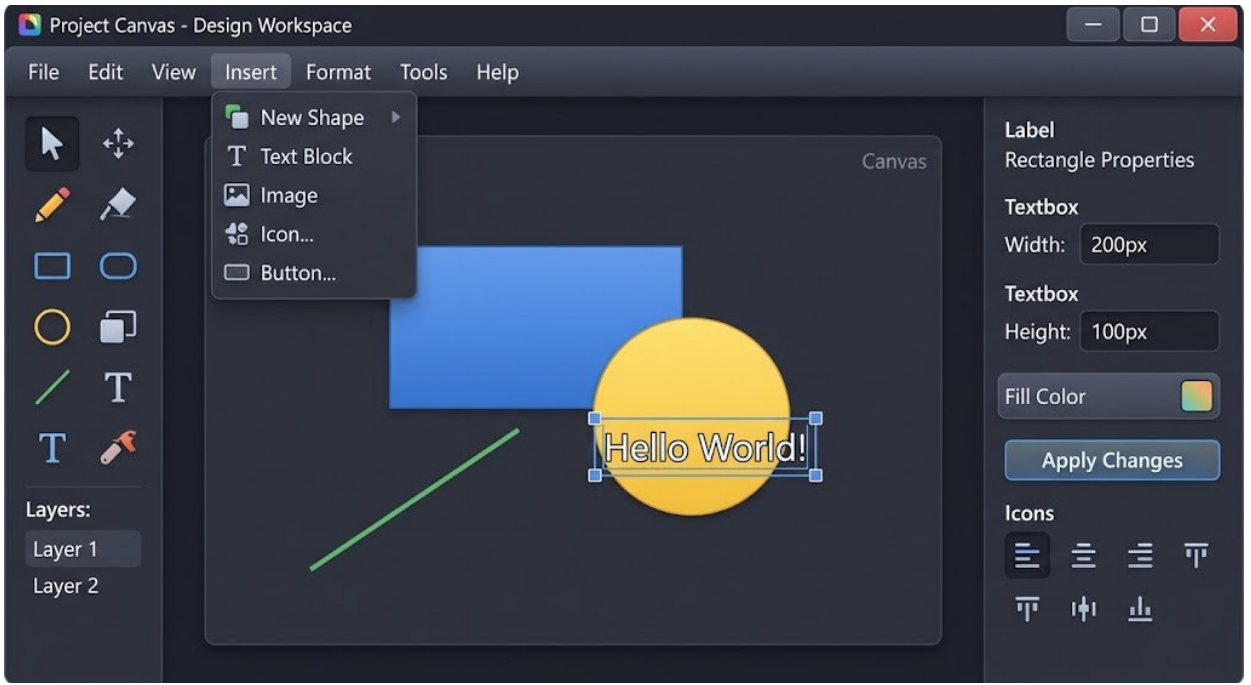
also typing, touching, tapping

https://en.wikipedia.org/wiki/Microsoft_Windows

Widget



Widget - Basic GUI element



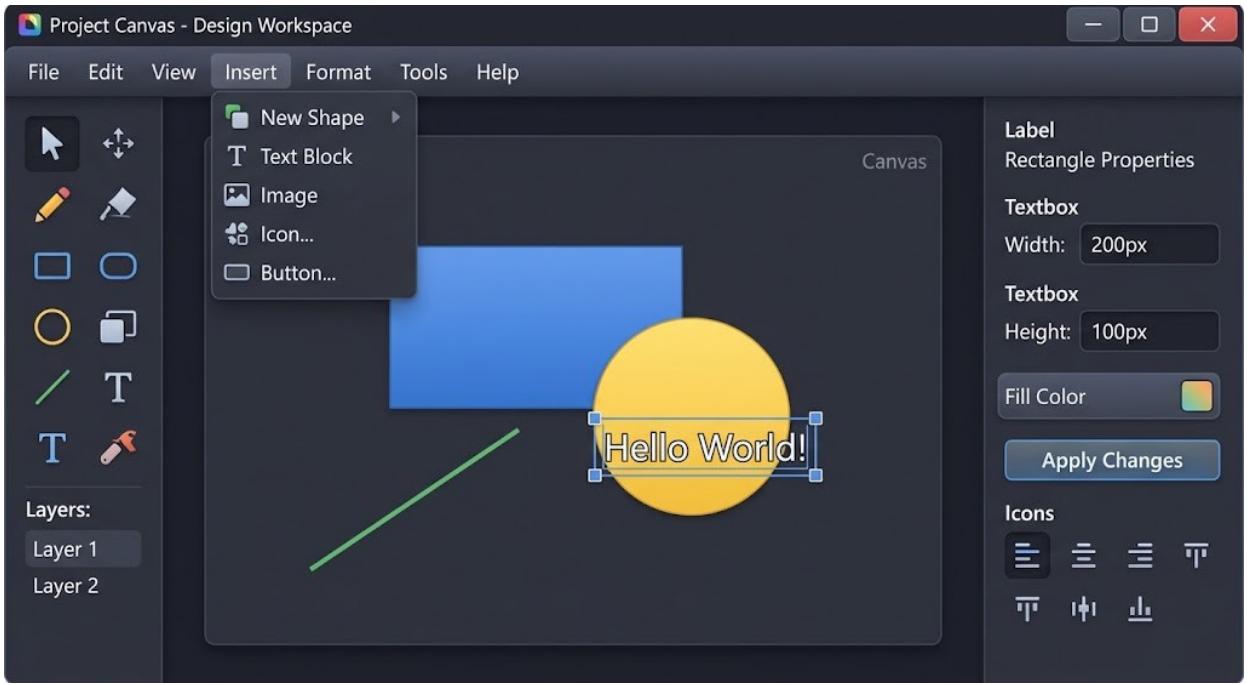
- Window
- Menu
- Icon
- Textbox
- Button
- Label
- Canvas

...

https://en.wikipedia.org/wiki/Microsoft_Windows

Event

Event - an action or occurrence in GUI



- Mouse click
- Mouse move
- Mouse enter
- Mouse leave
- Keyboard press
- Window resize

  ...

Event Handler



Event Handler - A Function or method that is called when an event occur

Tkinter



Python’s built-in library for creating GUI applications

A library is a collection of pre-written code that you can use in your own programs

e.g. math, os, mathplotlib, numpy

Create a Window



```
import tkinter

window = tkinter.Tk()  # create main window
```

Window settings



```
import tkinter

window = tkinter.Tk()  # create main window

window.title("My tk App") # set title

window.geometry("500x500") # set size
```

Event Listener Loop



```
import tkinter

window = tkinter.Tk()  # create main window

window.title("My tk App") # set title

window.geometry("500x500") # set size

window.mainloop()

# loop to wait for an event, e.g. user click
```

. . .



https://en.wikipedia.org/wiki/Microsoft_Windows

Add a Label



```
... # after create window

label = tkinter.Label(window, text="Hello")




... # before mainloop
```

Call layout manager



```
... # after create window

label = tkinter.Label(window, text="Hello")

label.pack()

# to manage this label (before showing it on window)


... # before mainloop
```

...

Add Textbox Entry



```
... # after create label
entry1 = tkinter.Entry(window)
entry1.pack()

entry2 = tkinter.Entry(window)
entry2.pack()

... # before mainloop
```

Add a Button



```
... # after create textboxes

button = tkinter.Button(window, text="Multiply")

button.pack()


... # before mainloop
```

Add Event Handler



```
def multiply():  # create a function
```

Get value from Textbox



```
def multiply():  # create a function
    num1 = float( entry1.get() )

    # get input value from textbox
```

Get value from Textbox



```
def multiply():  # create a function
    num1 = float( entry1.get() )
    num2 = float( entry2.get() )
```

Calculate result



```
def multiply():
    num1 = float( entry1.get() )
    num2 = float( entry2.get() )
    result = num1 * num2
```

Set Label



```
def multiply():
    num1 = float( entry1.get() )
    num2 = float( entry2.get() )
    result = num1 * num2
    label.config( text=result )
```

Set Event handler



```
# add command when create button

# button = tkinter.Button(window, text="Multiply")

button = tkinter.Button(window, text="Multiply",
command=multiply )
```

Set Event handler



```
# add command when create button

# button = tkinter.Button(window, text="Multiply")

button = tkinter.Button(window, text="Multiply",
command=multiply )

  # when this button is pressed
  # it will call multiply()
```

...

Try layout manager setting/option



```
pady - add vertical padding
```

Add veritcal padding



```
label.pack( pady=20 )

entry1.pack( pady=20 )

button.pack( pady=40 )
```

Add a Frame



```
entry_frame = tkinter.Frame(window)

entry_frame.pack(pady=20)
```

Add a Frame



```
window
│
├── label
├── entry1
├── entry2
└── button

# without frame
```

```
window
│
├── label
├── entry_frame
│   ├── entry1
│   └── entry2
└── button

# use frame to
manage layout
```

Add Entry to the Frame



```
entry1 = tkinter.Entry(entry_frame, width=10)
entry1.pack(side="left", padx=5)

entry2 = tkinter.Entry(entry_frame, width=10)
entry2.pack(side="left", padx=5)
```

Add Entry to the Frame



```
# entry1 = tkinter.Entry(window)

entry1 = tkinter.Entry(entry_frame, width=10)


# not to window directly
```

Try layout manager setting/option



```
pady - add vertical padding


grid layout
```

Try other widgets



https://tkdocs.com/tutorial/widgets.html

...

Assessment



Lecture	10%
Lab	10%
A1	10%
A2	10%
Midterm	30%
Final	30%

(maybe adjusted)

Assessment



A2 10%

app/game based on 2D array

Discussed with your group members?

. . .



010123138

Computer Programming



Check **Attendance..**



Review



Need a Volunteer



Class



A class is a blueprint/template for creating objects

An Object is an item/entity/คน/สัตว์/สิ่งของ

Specify what you are interested in

- Attributes (Data/state)
- Methods (Behaviour/action/calculation/function)

Big Picture



Consists of many item/entity/คน/สัตว์/สิ่งของ

- Balloon
- Cloud
- Cat, Dog, House, Tree, Car, Bike, ...

Solve the problem, and reuse later?

Create a class



```
class Balloon: # specify class name
    def __init__(self, x, y, r):  # specify constructor
        self.x = x
        self.y = y
        self.r = r
# similar concept to a function
# to construct/create object (a single balloon)
# create its attributes (x, y, r) from parameters
```

Add a method to a class



```
class Balloon: # specify class name
    def __init__(self, x, y, r):  # specify constructor
        ...
    def show(self):
        line(self.x, self.y, self.x, self.y + 100)
        ellipse(self.x, self.y, self.r, self.r)


# it will show/draw balloon
# using its own data/attributes (self.x, self.y, self.r)
```

Create Object



```
b = Balloon(400, 600, 50)


# this will call its constructor

# In Python __init__() will be called


# A Class is a template/blueprint of an object
```

Call a method



```
b = Balloon(400, 600, 50)


def setup():
    size(1000, 1000)
    b.show()


# graphic mode - should draw on screen :)
```

Add another method



```
    def move(self):
        if self.y > width / 2:
            self.y = self.y - 2
        else:
            self.y = self.y - 4


# only 1 parameter, "self"
```

Call move() to animate



```
def draw():
    background(255)
    b.show()
    b.move()


# do not need to change its data (attributes)?
# only call its methods, it knows what to do :)
```

...

Array of objects



```
balloons = []  # create an empty array


def setup():
    size(1000, 1000)
    ...
```

Array of objects



```
def setup():
    size(1000, 1000)
    x_coords = [100, 200, 400, 500, 600]
    y_coords = [900, 850, 600, 700, 950]


    i = 0
    while i < len(x_coords):
        balloons.append(Balloon(x_coords[i], y_coords[i],
50))
        i = i + 1
```

Array of objects



```
def draw():
    background(255)
    i = 0
    while i < len(balloons):
        balloons[i].show()
        balloons[i].move()
        i = i + 1
```

. . .

Class



A class is a blueprint/template for creating objects

Specify what you are interested in

- Attributes (Data/state)
- Methods (Behaviour/action/calculation/function)

User-defined



User-defined Variable

User-defined Function

User-defined



User-defined Variable

User-defined Function

User-defined Type (data type/Class)

Similar Idea in Biology

Kingdom
↓
Phylum
↓
Class
↓
Order
↓
Family
↓
Genus
↓
Species

Object-Oriented Programming (OOP)



OOP is a programming approach that organizes programs around objects

"Procedural programming" organizes a program as a sequence of procedures or functions

(what we did at the beginning)

4 Pillars of OOP



The four main concepts of Object-Oriented Programming

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

Encapsulation



Encapsulation – Bundling data and methods together and controlling access to them

Python does not enforce access control

(Python does not strictly enforce private/protected access like some languages do)

later we will try it with Java

Abstaction



Abstraction – Hiding unnecessary details
and showing only what is needed

We use the brake pedal to stop the car.

we don't need to know:

- How the braking system works internally
- Whether it uses disc brakes or drum brakes
- How brake pressure is generated

. . .

Inheritance



Inheritance – Creating a new class based on an existing class

Inheritance



Inheritance – Creating a new class based on an existing class

* New class will inherit (from an existing class)
    - ...
    - ...

Inheritance



Inheritance – Creating a new class based on an existing class

* New class will inherit (from an existing class)
    - Attributes
    - Methods

Inheritance



Inheritance – Creating a new class based on an existing class

* Subclass will inherit (from Superclass)
    - Attributes
    - Methods

Inheritance



Inheritance – Creating a new class based on an existing class

* class EsBalloon will inherit (from class Balloon)
    - Attributes
    - Methods

Wait for class EsBalloon

Inheritance



Different naming

* Superclass and subclass
* Base class and derived class
* Parent class and child class

...

Subclass



```
class EsBalloon( Balloon ):

    # EsBalloon is a sublcass of Balloon
    # for expanding and shinking balloon

    ...
```

Subclass



```
class EsBalloon( Balloon ):

    def __init__(self, x, y, r):

        # define the constuctor

        ...
```

Subclass



```
class EsBalloon( Balloon ):

    def __init__(self, x, y, r):

        super().__init__(x, y, r)

        # call the constuctor of the superclass

        ...
```

Subclass



```
class EsBalloon( Balloon ):

    def __init__(self, x, y, r):

        super().__init__(x, y, r)

        self.change = 1

        # add a new attribute
```

Add a method



```
def expand_shrink(self):
    self.r = self.r + self.change
    # to make it expand and shrink
    ...
```

Add a method



```
def expand_shrink(self):
    self.r = self.r + self.change

    if self.r >= 60:
        self.change = -1
    elif self.r <= 30:
        self.change = 1
```

Setup



```
def setup():
    size(1000, 1000)
    x_coords = [100, 200, 400, 500, 600]
    y_coords = [900, 850, 600, 700, 950]


      i = 0
      while i < len(x_coords):
          balloons.append( EsBalloon(x_coords[i], y_coords[i], 50) )
          i = i + 1
```

Draw



```
def draw():
    background(255)
    i = 0
    while i < len(balloons):
        balloons[i].show()
        balloons[i].move()
        balloons[i].expand_shrink()
        i = i + 1
```

Draw



```
def draw():
    background(255)
    i = 0
    while i < len(balloons):
        balloons[i].show()
        balloons[i].move()
        # balloons[i].expand_shrink() # include it in move()?
        i = i + 1
```

Inheritance



Inheritance – Creating a new class based on an existing class

* Subclass will inherit (from Superclass)
    - Attributes
    - Methods

Method Override



Inheritance – Creating a new class based on an existing class

* Subclass will inherit (from Superclass)
  - Attributes
  - Methods

Can **override** a method of the superclass

  - move() in EsBalloon may include expand_shrink()

Method Override



```
class EsBalloon(Balloon):

    ...

    def move(self):
        # override move() in the superclass
        # so it will not do the same thing

        ...
```

Method Override



```
class EsBalloon(Balloon):

    ...

    def move(self):   # override move() in the superclass

        super().move()   # call the superclass method

        self.expand_shrink()   # call its own method
```

. . .

Text Mode



```
Person

# subclass?
```

Text Mode



```
Person

    Student
```

Text Mode



```
Person

    Student
```

Text Mode



```
Person
    Student

    Employee
```

Text Mode



```
Person
    Student

    Employee


// show_info() ?
```

Text Mode



```
Person
    Student

    Employee


// show_info() ? call superclass method
```

Text Mode



```
Person
    Student

    Employee

// getBMI() ? call superclass method
```

Example Classes



```
Person
    Student

    Employee


        Manager
```

Example Classes



```
Person
    Student

    Employee


        Manager


// staff discount?
```

...

Polymorphism



Polymorphism – Many shapes/forms

Polymorphism



Polymorphism – Many shapes/forms

- Method overriding
- Subtype

Polymorphism



Polymorphism – Many shapes/forms

- Method overriding

  A method with the same name
  can have different behaviour

Polymorphism



Polymorphism – Many shapes/forms

- Method overriding

  A method with the same name
  can have different behaviour

move() in EsBalloon can be different from its superclass

Polymorphism



Polymorphism – Many shapes/forms

- Method overriding
- Subtype

  EsBalloon is a Balloon?

  Person can be a student?

  have the same set of methods?

Polymorphism



Polymorphism – Many shapes/forms

- Method overriding
- Subtype

  EsBalloon is a Balloon?

  Person can be a student?

. . .

Old Syntax



Python-mode Processing

```
super().__init__(x, y, r)

# does not work, must change to

super(EsBalloon, self).__init__(x, y, r)


# get the superclass of EsBalloon, then call its constructor
```

Old Syntax



Python-mode Processing

```
class Balloon( object ):

    def __init__(self, x, y, r):
      self.x = x
      self.y = y
      self.r = r


# Specify object as the superclass of Balloon
```

...



Assessment



Lecture	10%
Lab	10%
A1	10%
A2	10%
Midterm	30%
Final	30%

(maybe adjusted)

Assessment



Lecture 10% consists of

- After class quizzes (individual)
- Task 1 (group work)
- Task 2 (group work)

Assessment



Lecture 10% consists of

- Task 1 (group work)
    - find exercises (coding question) and write solution
    - create quizzes (google form)

w1 for group 1, w2 for group 2, ...

Assessment



Lecture 10% consists of

- Task 2 (group work)
  - find 5 example classes from other textbooks in the library (may not in Python)
  - note their attributes and methods

Assessment



Lecture 10% consists of

- Task 2 Part 2 (group work)

  - find 5 example class inheritance from other textbooks in the library (may not in Python)

  - note their attributes and methods

(create slides and submit by Friday, and every member prepare to present next week)

. . .

010123138

Computer Programming



Check **Attendance..**



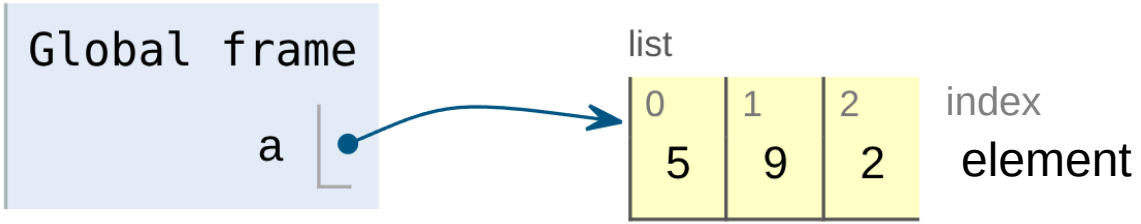
Review



Array of variables



```
a = [ 5, 9, 2 ]
```




```
# Python use "list" (similar but not the same as array)

# from this example: size or length is 3
# index start from 0 (up to 2)
```

Function and Array



```
def print_all(a):
    i = 0  # index
    while ( i < len(a)):  # len(a) is length of a
        print( a[i] )     # print element at i
        i = i + 1


a = [ 5, 9, 2 ]
print_all(a)
```

2D Array



```
# 1D array (list of elements)
a = [ 3, 8, 2 ]


# 2D array (list of lists)
m = [ [ 5, 7, 9 ], [ 2, 4, 6 ], [ 8, 3, 1 ] ]
#          m[0]         m[1]         m[2]
```

2D Array



```
# 2D array (list of lists)

m = [ [ 5, 7, 9 ], \
      [ 2, 4, 6 ], \
      [ 8, 3, 1 ] ]


print( m[0][0] )  # 5   use 2 indexes (row,col)

print( m[0][1] )  # 7

print( m[1][0] )  # 2
```

2D Array



```
# 2D array (list of lists)

# use 2 indexes (row,col)


m[1][2] = 7

m[2][2] = 5
```

Array of Arrays



```
def print_all(a):       # same as before (for 1D array)
    i = 0  # index
    while ( i < len(a)):
        print( a[i] )
        i = i + 1


print_all( m[1] )  # send an array, m[1]
```

Array of Arrays



```
# print every element

print_all( m[0] )

print_all( m[1] )

print_all( m[2] )


# need a loop?
```

Array of Arrays



```
def print_2D(m):
    i = 0  # index
    while ( i < len(m)):   # loop
        print_all( m[i] )  # another loop?
        i = i + 1


print_2D( m )  # send an array, m
```

Nested Loop



```
def print_2Dn(m):
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( m[i] )):   # nested loop
            print( m[i][j] )
            j = j + 1
        i = i + 1
```

Sum



```
def sum_2Dn(m):
    sum = 0
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( m[i] )):   # inner loop
            sum = sum + m[i][j]
            j = j + 1
        i = i + 1
    return sum
```

Count



```
def count_2Dn(m , x):
    count = 0
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( m[i] )):   # inner loop
            if (m[i][j] == x):
                count = count + 1
            j = j + 1
        i = i + 1
    return count
```

Matrix



```
A = [[1, 2],\
     [3, 4]]

B = [[5, 6],\
     [7, 8]]

# can use 2D array?
```

Add 2 Matrices



```
def addm(A, B):
    result = []
    i = 0
    while i < len(A):  # loop every row
        row = []
        j = 0
        while j < len(A[i]): # loop every column
            row.append( A[i][j] + B[i][j] )
            j = j + 1
        result.append( row )
        i = i + 1
    return result
```

Add 2 Matrices



```
A = [[1, 2],\
     [3, 4]]


B = [[5, 6],\
     [7, 8]]
print( addm(A, B) )
# Try and see on pythontutor.com
```

Need a Volunteer



Problem Solving



How to organise and reuse code?

Problem Solving



- **variables** (single, array)

- **functions** (statements, conditionals, loops)

Problem Solving



- **variables** (single, array)

- **functions** (statements, conditionals, loops)

  divide into several functions
  (draw_balloon(), draw_house())

Problem Solving



- **variables** (single, array)

- **functions** (statements, conditionals, loops)

  to draw ballon in another program

Floating Balloon



```
# What data are needed
```

Floating Balloon



```
# What data are needed

# x , y, size
```

Floating Balloon



```
# What data are needed

# x , y, size

# how we store them
```

Create a class



```
class Balloon:

    ...



# to store data of a single balloon
```

Create a class



```
class Balloon: # specify class name

    ...
```

Create a class



```
class Balloon: # specify class name

    def __init__( ... ):  # specify constructor

        ...


# similar concept to a function

# to construct/create object (a single balloon)
```

Create a class



```
class Balloon: # specify class name
    def __init__(self, x, y, r):  # specify constructor
        ...


# similar concept to a function
# to construct/create object (a single balloon)
# need x, y, r as parameter (skip self for now :)
```

Create a class



```
class Balloon: # specify class name
    def __init__(self, x, y, r):  # specify constructor
        self.x = x
        self.y = y
        self.r = r
# similar concept to a function
# to construct/create object (a single balloon)
# create its attributes (x, y, r) from parameters
```

Create Object



```
b = Balloon(400, 600, 50)


# this will call its constructor

# In Python __init__() will be called


# A Class is a template/blueprint of an object
```

Create Object



```
b = Balloon(400, 600, 50)


# this will call its constructor

# In Python __init__() will be called


# A Class is a template/blueprint of an object
```

May print it?



```
b = Balloon(400, 600, 50)


def setup():
    size(1000, 1000)
    print(b)
```

May print it?



```
b = Balloon(400, 600, 50)


def setup():
    size(1000, 1000)
    print(b)


# graphic mode - should draw on screen :)
```

...

Add a method to a class



```
class Balloon: # specify class name
    def __init__(self, x, y, r):  # specify constructor
        ...
    def show(self):  # it will show/draw balloon
        ...
# similar concept to a function
# self -> itself     (many languages use "this")

# using its own data/attributes (x, y, r)
```

Add a method to a class



```
class Balloon: # specify class name
    def __init__(self, x, y, r):  # specify constructor
        ...
    def show(self):
        line(self.x, self.y, self.x, self.y + 100)
        ellipse(self.x, self.y, self.r, self.r)


# it will show/draw balloon
# using its own data/attributes (self.x, self.y, self.r)
```

Assign/Use attributes



```
class Balloon: # specify class name
    def __init__(self, x, y, r):  # specify constructor
        self.x = x
        self.y = y
        self.r = r
    ...
# show() will show/draw balloon
# using its own data/attributes (self.x, self.y, self.r)
# assign when created an object, constructor is called
```

Call a method



```
b = Balloon(400, 600, 50)


def setup():
    size(1000, 1000)
    b.show()


# graphic mode - should draw on screen :)
```

...

Add another method



```
    def move(self):
        if self.y > width / 2:
            self.y = self.y - 2
        else:
            self.y = self.y - 4


# only 1 parameter, "self"
```

Call move() to animate



```
def draw():
    background(255)
    b.show()
    b.move()
```

Call move() to animate



```
def draw():
    background(255)
    b.show()
    b.move()


# do not need to change its data (attributes)?
# only call its methods
```

. . .

Array of objects



```
balloons = []  # create an empty array


def setup():
    size(1000, 1000)
    ...
```

Array of objects



```
def setup():
    size(1000, 1000)
    x_coords = [100, 200, 400, 500, 600]
    y_coords = [900, 850, 600, 700, 950]

    i = 0
    while i < len(x_coords):
        balloons.append(Balloon(x_coords[i], y_coords[i], 50))
        i = i + 1
```

Array of objects



```
def draw():
    background(255)
    i = 0
    while i < len(balloons):
        balloons[i].show()
        balloons[i].move()
        i = i + 1
```

...

Floating Cloud



```
# What data are needed

# x , y, size


# To try
```

Class



What you are interested in

- Attributes (Data/state)
- Methods (Behaviour/action/calculation/function)

Big Picture



Consists of many item/entity/คน/สัตว์/สิ่งของ

- Balloon
- Cloud
- Cat, Dog, House, Tree, ...

Big Picture



Consists of many item/entity/คน/สัตว์/สิ่งของ

- Balloon
- Cloud
- Cat, Dog, House, Tree, Car, Bike, ...

Solve the problem, and reuse later?

...

Python Built-in Classes



- int, float, bool, complex
- list, str, set, dict
- file (file-like objects)

(try then find more)

Class File in Python



For reading and writing text file

Class File in Python



```
f = open("output.txt", "w")   # open file for writing
```

Class File in Python



```
f = open("output.txt", "w")   # open file
f.write( "Hello, everybody" ) # call method write()
```

Class File in Python



```
f = open("output.txt", "w")   # open file
f.write( "Hello, everybody" ) # call method write()


# f is an object
# write( ) is a method
```

Class File in Python



```
f = open("output.txt", "w")   # open file
f.write( "Hello, everybody" ) # call method write()
f.close()


# close( ) is a method


# do not forget to close the file after writing
```

Class File in Python



```
f = open("output.txt", "w")   # open file
f.write( "Hello, everybody" ) # call method write()
f.close()


# different from print() ?
```

File name and Path



path is a string that spectify location of a file or directory

e.g.

c:\doc.txt

c:\windows

d:\home\example\

Current working directory



```
# Where your program is running
```

Current working directory



```
# Where your program is running


import os

print( os.getcwd( ) )


# cwd - current working directory
```

Change directory



```
import os

print( os.chdir( "c:\\home" ) )   # may need escape \
```

Change directory



```
import os

print( os.chdir( "c:\\home" ) )   # may need escape \


print( os.getcwd( ) ) # cwd will change to c:\home
```

List directory



```
import os
print( os.getcwd( ) )


print( os.listdir( ) ) # list of files and directories in cwd


e.g.
   ['example.txt', 'program', 'doc', 'lab.py']
```

Boolean functions



```
import os

print( os.path.exists("sample") ) # the path exists ?
```

Boolean functions



```
import os

print( os.path.exists("sample") ) # the path exists ?


print( os.path.isfile("sample") ) # is it a file?


print( os.path.isdir("sample") ) # is it a directory?
```

File Size



```
import os

file_size = os.path.getsize( "output.txt" )
```

File Size



```
import os

file_size = os.path.getsize( "output.txt" )


print( "File Size is :", file_size, "bytes" )
```

Abstaction



Hide complex implementation details

- how the operating system represents files
- how filesystem metadata is accessed
- how paths are handled

Works other Operating Systems?

Abstaction



Hide complex implementation details

- how the operating system represents files
- how filesystem metadata is accessed
- how paths are handled

Compare to using a TV remote / drive a car

Encapsulation



Hide data, and only allow to access via methods

Encapsulation



Hide data, and only allow to access via methods

Should not access Balloon x,y positions directly?

Encapsulation



Hide data, and only allow to access via methods

Should not access Balloon x,y positions directly?

use move() , show()

...

Create your own



- my_complex

(from array/list?)

Create your own



- my_complex, my_set

(from array/list?)

...

Prepare Python Library



- Turtle
- Tkinter
- Matplotlib

Try sample code



```
import turtle

t = turtle.Turtle()

t.forward(100)

t.left(90)



t.forward(100)

t.right(90)



t.forward(100)

t.left(90)



t.forward(100)

t.right(90)



t.forward(100)

turtle.done()
```

Try sample code



```
import tkinter as tk
root = tk.Tk()

canvas = tk.Canvas(root, width=500, height=500)
canvas.pack()

canvas.create_oval(200, 50, 400, 250, fill="orange")
canvas.create_line(300, 250, 300, 450)

root.mainloop()
```

Try sample code



```
import matplotlib.pyplot as plt

# 4 weeks × 7 days

temperatures = [

    [30, 31, 32, 30, 30, 31, 33],  # Week 1

    [29, 30, 31, 32, 31, 30, 29],  # Week 2

    [31, 32, 33, 34, 32, 31, 30],  # Week 3

    [30, 29, 30, 30, 29, 30, 32]   # Week 4

]

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

for week in temperatures:

    plt.plot(days, week)

plt.xlabel("Day")

plt.ylabel("Temperature (C)")

plt.title("Temperature for 4 Weeks")

plt.legend(["Week 1", "Week 2", "Week 3", "Week 4"])

plt.show()
```

. . .

Assessment



Lecture	10%
Lab	10%
A1	10%
A2	10%
Midterm	30%
Final	30%

(maybe adjusted)

Assessment



Lecture 10% consists of

- After class quizzes (individual)
- Task 1 (group work)
- Task 2 (group work)

Assessment



Lecture 10% consists of

- Task 1 (group work)
    - find exercises (coding question) and write solution
    - create quizzes (google form)

w1 for group 1, w2 for group 2, ...

Assessment



Lecture 10% consists of

- Task 2 (group work)

  - find 5 example classes from other textbooks in the library

    (may not in Python)

Assessment



Lecture 10% consists of

- Task 2 (group work)
  - find 5 example classes from other textbooks in the library (may not in Python)
  - note their attributes and methods

(create slides and submit by Friday, and every member prepare to present next week)

...

010123138

Computer Programming



Check **Attendance..**



Review



Non-Volatile data



File on Storage (e.g. HDD/SSD)

- Persists without power, can be reused later

- Slower read/write speed
  (compared to RAM)

File Types



- Text File (e.g. .txt, .csv, .json, .html)
  They are human-readable, stored as characters (string)

- Binary File (e.g. .jpg, .mp3, .exe)
  for program to read

File Operations



```
# Open a text file for reading ("r")
file = open("example.txt", "r")


# Read the entire content and print it
content = file.read()  # call a method
print( content )  # Content is a string


# Always close the file when finished
file.close()
```

with statement



```
with open("example.txt", "r") as file:
    content = file.read()
    print( content )

# Recommended Way
# The file will be automatically closed
```

while the line is not empty



```
with open("example.txt", "r") as f:
    line = f.readline()    # Read a line from file
    while line :
        print( line )        # print it
        ...
```

while the line is not empty



```
with open("example.txt", "r") as f:
    line = f.readline()    # Read a line from file
    while line :
        print( line )          # print it
        line = f.readline() # read the next line

# Loop condition :  line is not empty

# not the end of file (EOF)
```

for (each) loop



```
with open("example.txt", "r") as file:
    for line in file:    # for each line in file
```

Print line-by-line



```
with open("example.txt", "r") as file:
    for line in file:     # for each line in file
        print( line )     # print it


# Loop condition ?
```

Process line-by-line



```
a = [] # start from empty list
with open("example.txt", "r") as file:
    for line in file:
        num = int( line )  # convert to int
        a.append( num )
print( a )  # result array (read values from file)

# [5, 2, 7]
```

```
5
2
7
```

Process a line



```
a = [] # start from empty list
with open("example.txt", "r") as file:
    for line in file:
        # process and append to array


# string exercise?

# [5, 2, 7]
```

```
5,2,7
```

Process a line



- Find a word in file

```
# string exercise?
```

Write to file



```
with open("output.txt", "w") as file:
    file.write("Hello Python\n")
    file.write("Welcome to Thailand\n")


# similar to print()?


# must add newline (\n) manually
```

Write values of Array



```
with open("output.txt", "w") as file:
    ...


# exercise
```

Invalid data from file



- String processing?

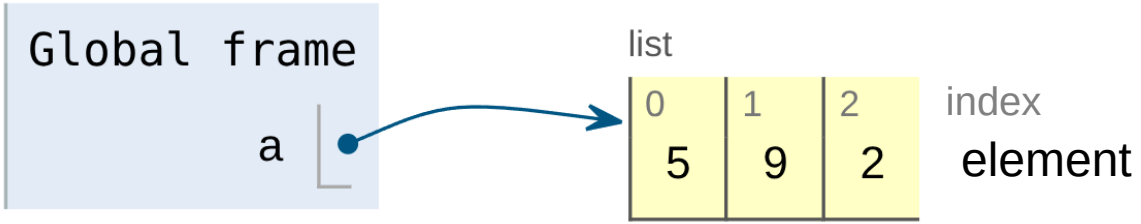
  isalpha(), isdigit(), is..?

...

Array of variables



```
a = [ 5, 9, 2 ]
```




```
# Python use "list" (similar but not the same as array)

# from this example: size or length is 3
# index start from 0 (up to 2)
```

Print elements of Array



```
a = [ 5, 9, 2 ]

i = 0  # index

while ( i < len(a)):  # len(a) is length of a

    print( a[i] )     # print element at i

    i = i + 1
```

Function and Array



```
def print_all(a):
    i = 0  # index
    while ( i < len(a)):  # len(a) is length of a
        ...               # print element at i
        i = i + 1


a = [ 5, 9, 2 ]
...  # call the function
```

Function and Array



```
def print_all(a):
    i = 0  # index
    while ( i < len(a)):  # len(a) is length of a
        print( a[i] )     # print element at i
        i = i + 1


a = [ 5, 9, 2 ]
print_all(a)
```

Function and Array



```
a = [ 5, 9, 2 ]

print_all(a)


b = [ 6, 1, 8 ]

print_all(b)
```

Function and Array



```
def add_all(a, x):
    i = 0  # index
    while ( i < len(a)):  # len(a) is length of a
        a[i] = a[i] + x   # add x to a[i]
        i = i + 1


a = [ 5, 9, 2 ]
add_all(a, 3)              # pass by reference?
print_all(a)
```

Function and Array



```
a = [ 5, 9, 2 ]

add_all(a, 3)

print_all(a)        # can change elements in a


b = [ 6, 1, 8 ]

add_all(b, 2)

print_all(b)        # can change elements in b
```

Pass by value



```
def add_x(a, x):
    a = a + x


a = 5
add_x(a, 3)
print(a)
```

Pass by value



```
def add_x(a, x):
    a = a + x


a = 5
add_x(a, 3)
print(a)              # value of a is unchanged
```

Pass by value



```
def add_x(a, x):
    a = a + x


b = 2
add_x(b, 7)
print(b)            # value of b is unchanged
```

Pass by value



```
def add_x(a, x):
    a = a + x


b = 2
add_x(b, 7)          # value of b is copied to a
print(b)             # b is unchanged
                     # try on pythontutor.com
```

Need a Volunteer



2D Array



```
# 1D array (list of elements)
a = [ 3, 8, 2 ]


# 2D array (list of lists)
m = [ [ 5, 7, 9 ], [ 2, 4, 6 ], [ 8, 3, 1 ] ]
```

2D Array



```
# 1D array (list of elements)
a = [ 3, 8, 2 ]


# 2D array (list of lists)
m = [ [ 5, 7, 9 ], [ 2, 4, 6 ], [ 8, 3, 1 ] ]
#          m[0]         m[1]         m[2]
```

2D Array



```
# 2D array (list of lists)

m = [ [ 5, 7, 9 ], \
      [ 2, 4, 6 ], \
      [ 8, 3, 1 ] ]
```

2D Array



```
# 2D array (list of lists)

m = [ [ 5, 7, 9 ], \
      [ 2, 4, 6 ], \
      [ 8, 3, 1 ] ]
```

2D Array



```
# 2D array (list of lists)

m = [ [ 5, 7, 9 ], \
      [ 2, 4, 6 ], \
      [ 8, 3, 1 ] ]


print( m[0][0] )

print( m[0][1] )

print( m[1][0] )
```

2D Array



```
# 2D array (list of lists)

m = [ [ 5, 7, 9 ], \
      [ 2, 4, 6 ], \
      [ 8, 3, 1 ] ]


print( m[0][0] )  # 5   use 2 indexes (row,col)

print( m[0][1] )  # 7

print( m[1][0] )  # 2
```

2D Array



```
# 2D array (list of lists)

m = [ [ 5, 7, 9 ], \
      [ 2, 4, 6 ], \
      [ 8, 3, 1 ] ]


print( m[1][1] )      # use 2 indexes (row,col)

print( m[2][2] )

print( m[1][2] )
```

2D Array



```
# 2D array (list of lists)

# use 2 indexes (row,col)


m[1][2] = 7

m[2][2] = 5
```

2D Array



```
# 1D array (list of elements)
a = [ 3, 8, 2 ]


# 2D array (list of lists)
m = [ [ 5, 7, 9 ], [ 2, 4, 6 ], [ 8, 3, 1 ] ]
#          m[0]         m[1]         m[2]
```

Array of Arrays



```
def print_all(a):       # same as before (for 1D array)
    i = 0  # index
    while ( i < len(a)):
        print( a[i] )
        i = i + 1


print_all( m[1] )  # send an array, m[1]
```

Array of Arrays



```
# print every element

print_all( m[0] )

print_all( m[1] )

print_all( m[2] )
```

Array of Arrays



```
# print every element

print_all( m[0] )

print_all( m[1] )

print_all( m[2] )


# need a loop?
```

Array of Arrays



```
def print_2D(m):
    i = 0  # index
    while ( i < len(m)):
        ...
        i = i + 1


print_2D( m )  # send an array, m
```

Array of Arrays



```
def print_2D(m):
    i = 0  # index
    while ( i < len(m)):
        print_all( m[i] )  # send an array, m[i]
        i = i + 1


print_2D( m )  # send an array, m
```

Array of Arrays



```
def print_2D(m):
    i = 0  # index
    while ( i < len(m)):   # loop
        print_all( m[i] )  # another loop?
        i = i + 1


print_2D( m )  # send an array, m
```

Nested Loop



```
def print_2Dn(m):
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( ... )):   # inner loop
            print( ... )
            j = j + 1
        i = i + 1
```

Nested Loop



```
def print_2Dn(m):
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( m[i] )):   # nested loop
            print( m[i][j] )
            j = j + 1
        i = i + 1
```

Sum



```
def sum_2Dn(m):
    ...
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( m[i] )):   # inner loop
            ...
            j = j + 1
        i = i + 1
    ...
```

Sum



```
def sum_2Dn(m):
    sum = 0
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( m[i] )):   # inner loop
            sum = sum + m[i][j]
            j = j + 1
        i = i + 1
    return sum
```

Sum



```
m = [ [ 5, 7, 9 ], [ 2, 4, 6 ], [ 8, 3, 1 ] ]


print( sum_2Dn(m) )
```

Count



```
def count_2Dn(m , x):
    ...
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( m[i] )):   # inner loop
            ...
            ...
            j = j + 1
        i = i + 1
    ...
```

Count



```
def count_2Dn(m , x):
    count = 0
    i = 0  # index
    while ( i < len(m)):   # outer loop
        j = 0
        while ( j < len( m[i] )):   # inner loop
            if (m[i][j] == x):
                count = count + 1
            j = j + 1
        i = i + 1
    return count
```

...

Algorithms



1D Array

- find max/min/sum/avg
- find element/index (linear/binary search)
- count occurrences
- check array is sorted
- sort array (bubble sort?)

Algorithms



2D Array

- find max/min/sum/avg (all/row/col)
- find element/index (all/row/col)
- count occurrences (all/row/col)
- check array is sorted ?

Algorithms



- Can we use a 2D array

  to keep the history of a 1D array being sorted?

...

Matrix



```
A = [[1, 2],\
     [3, 4]]

B = [[5, 6],\
     [7, 8]]

# can use 2D array?
```

Add 2 Matrices



```
def addm(A, B):
    result = []
    i = 0
    while i < len(A):  # loop every row
        row = []
        j = 0
        while j < len(A[i]): # loop every column
            row.append( A[i][j] + B[i][j] )
            j = j + 1
        result.append( row )
        i = i + 1
    return result
```

Add 2 Matrices



```
A = [[1, 2],\
     [3, 4]]


B = [[5, 6],\
     [7, 8]]
print( addm(A, B) )
# Try and see on pythontutor.com
```

Multi-dimension Array



```
# 2D array (list of lists)

# use 2 indexes (row,col)



# 3D array ...
# use ... indexes
```

Multi-dimension Array



```
# 2D array (list of lists)

# use 2 indexes (row,col)



# 3D array (list of 2D array :)
# use 3 indexes ( i, j, k ?)
```

Multi-dimension Array



```
# 2D array (list of lists)

# use 2 indexes (row,col)



# 3D array (list of 2D array :)
# use 3 indexes ( i, j, k ?)



# 4D ?
```

Multi-dimension Array



```
# do not need to be 3 x 3 x 3

# 2 x 3 x 3 ?
```

...

010123138

Computer Programming



Check **Attendance..**



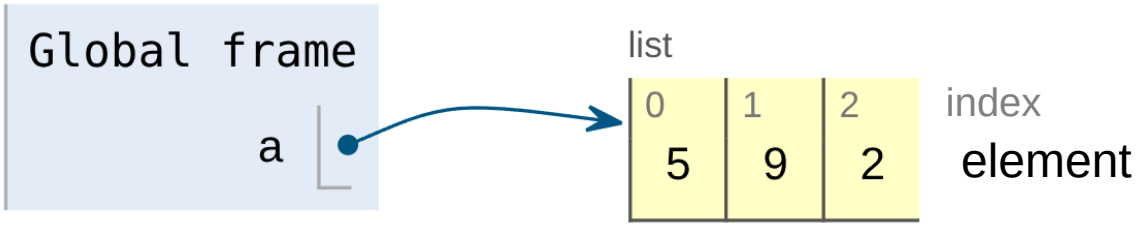
Review



Array of variables



```
a = [ 5, 9, 2 ]
```




```
# Python use "list" (similar but not the same as array)

# from this example: size or length is 3
# index start from 0 (up to 2)
```

Print elements of Array



```
a = [ 5, 9, 2 ]
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    print( a[i] )     # print element at i
    i = i + 1
```

Algorithms



- find max
- check array is sorted
- sort array?
- calculate factorial

Factorial



- using loop
- using recursive function

see on pythontutor.com

String



```
s = "Hello"  # string of characters


print(s)     # display string in text mode
```

String



```
s = "Hello"  # similar to array of characters
i = 0
while(i < len(s)):
    print( s[i] )
    i = i + 1
```




```
# In Python, a character is a string of length one

# but not in Java or C
```

Escape Characters



```
print("Hello\nWorld")
```

```
Hello
World
```

```
print("Python\tProgramming")
```

```
Python  Programming
```

```
print("He said \"Hello\"")
```

```
He said "Hello"
```

String Concatenation (+)



```
s = "Hello"
t = "Joe"
print( s + t )
```

String Concatenation (+)



```
s = "Hello"
t = ""
i = 0
while(i < len(s)):
    t = t + str(i)
    i = i + 1
print( t )
```

01234

String Concatenation (+)



```
s = "Hello"
t = ""
i = 0
while(i < len(s)):
    t = t + str(i) + "\t" + s[i] + "\n"
    i = i + 1
print( t )
```

```
0   H
1   e
2   l
3   l
4   o
```

Display String (graphic-mode)



```
def draw():
    fill(0)          # text color
    textSize(100)    # text size :)
    text("ABC", mouseX, mouseY)
```

Array is mutable



```
a = [ 5, 9, 2 ]

a[0] = 7
```

Can change element in array

String is immutable



```
s = "Hello"

s[0] = "J"   # Error

# TypeError: 'str' object does not support item assignment

# Cannot change element in array
```

Create a new string



```
s = "Hello"

s = "J" + s[4] + s[1]

# Assign it to the same variable

# Similar to earlier example
# t = t + str(i) + "\t" + s[i] + "\n"
```

String Method (Function)



```
s = "3"

print( s.isalnum() )


# See
# https://py.processing.org/reference/string
# https://docs.python.org/3/library/stdtypes.html#string-methods
# (not the main focus)
```

True

Question?



Review Questions



. . .

Volatile data



Variables, Array, String

- Temporary information stored in RAM
- Lost when program is closed
  or power is lost

Non-Volatile data



File on Storage (e.g. HDD/SSD)

- Persists without power, can be reused later
- Slower read/write speed
  (compared to RAM)

File Types



- Text File (e.g. .txt, .csv, .json, .html)
  They are human-readable, stored as characters (string)

- Binary File (e.g. .jpg, .mp3, .exe)
  for program to read

File Operations



```
# Open a text file for reading ("r")
file = open("example.txt", "r")

# later, try other file modes: r, w, a
```

File Operations



```
# Open a text file for reading ("r")
file = open("example.txt", "r")

# Read the entire content and print it
content = file.read()  # call a method
print( content )  # Content is a string
```

File Operations



```
# Open a text file for reading ("r")
file = open("example.txt", "r")

# Read the entire content and print it
content = file.read()  # call a method
print( content )  # Content is a string

# Always close the file when finished
file.close()
```

with statement



```
with open("example.txt", "r") as file:
    content = file.read()
    print( content )

# Recommended Way
# The file will be automatically closed
```

a variable named f



```
with open("example.txt", "r") as f:
    content = f.read()
    print( content )

# use file to make it clear
```

Read a line



```
with open("example.txt", "r") as f:
    line = f.readline()    # Read a line from file
```

Print a line



```
with open("example.txt", "r") as f:
    line = f.readline()    # Read a line from file
    print( line )          # print it
```

Read more lines?



```
with open("example.txt", "r") as f:
    line = f.readline()    # Read a line from file
    print( line )          # print it
     ...
```

while the line is not empty



```
with open("example.txt", "r") as f:
    line = f.readline()    # Read a line from file
    while line :
        print( line )        # print it
        ...
```

while the line is not empty



```
with open("example.txt", "r") as f:
    line = f.readline()    # Read a line from file
    while line :
        print( line )          # print it
        line = f.readline() # read the next line
```

while the line is not empty



```
with open("example.txt", "r") as f:
    line = f.readline()    # Read a line from file
    while line :
        print( line )          # print it
        line = f.readline() # read the next line

# Loop condition :  line is not empty

# not the end of file (EOF)
```

...

for (each) loop



```
with open("example.txt", "r") as file:
    for line in file:    # for each line in file
```

for (each) loop



```
with open("example.txt", "r") as file:
    for line in file:     # for each line in file
        print( line )     # print it
```

Print line-by-line



```
with open("example.txt", "r") as file:
    for line in file:     # for each line in file
        print( line )     # print it

# Loop condition ?
```

Process line-by-line



```
a = [] # start from empty list
with open("example.txt", "r") as file:
    for line in file:
        ...
        a.append( num )


# [5, 2, 7]
```

```
5
2
7
```

Process line-by-line



```
a = [] # start from empty list
with open("example.txt", "r") as file:
    for line in file:
        num = int( line )  # convert to int
        a.append( num )


# [5, 2, 7]
```

```
5
2
7
```

Process line-by-line



```
a = [] # start from empty list
with open("example.txt", "r") as file:
    for line in file:
        num = int( line )  # convert to int
        a.append( num )
print( a )  # result array (read values from file)

# [5, 2, 7]
```

```
5
2
7
```

Process line-by-line



- Find max/min/avg value in file

```
# need to create array?
```

```
5
2
7
12
4
9
```

Process a line



```
a = [] # start from empty list
with open("example.txt", "r") as file:
    for line in file:
        # process and append to array


# string exercise?

# [5, 2, 7]
```

5,2,7

Process a line



- Find a word in file

```
# string exercise?
```

...

Write to file



```
with open("output.txt", "w") as file:
    file.write("Hello Python\n")
    file.write("Welcome to Thailand\n")

# similar to print()?
```

Write to file



```
with open("output.txt", "w") as file:
    file.write("Hello Python\n")
    file.write("Welcome to Thailand\n")

# similar to print()?

# must add newline (\n) manually
```

Append to file



```
with open("output.txt", "a") as file:
    file.write("ECE\n")
    file.write("KMUTNB\n")

# add to the end of file
```

Write values of Array



```
with open("output.txt", "w") as file:
    ...


# exercise
```

. . .

File not found



```
with open("example.txt", "r") as file:
    for line in file:
        print(line)
```

```
FileNotFoundError: No such file or directory: 'example.txt'
```

Try and catch exception



```
try:
    with open("example.txt", "r") as f:
        for line in file:
            print(line)

except FileNotFoundError:
    print("Error: example.txt is not found")
```

Invalid data from file



- String processing?

  isalpha(), isdigit(), is..?

...

A1



Save/load A1 data from file

010123138

Computer Programming



Check **Attendance..**



Review



Sum from 1 to n



```
n = 5
sum = 0
count = 1

while count <= n:
    sum = sum + count
    count = count + 1

print("Sum =", sum)
```

Define a function



```
def sum_1_to_n( n ):
    sum = 0
    count = 1

    while count <= n:
        sum = sum + count
        count = count + 1
    return sum
```

Call the function



```
n = 5
print("Sum from 1 to",n,"=", sum_1_to_n( n ))

n = 10
print("Sum from 1 to",n,"=", sum_1_to_n( n ))
```

define main()



```
# do not need global variables
# try and see on pythontutor.com

def main():
    n = 5
    print("Sum from 1 to",n,"=", sum_1_to_n( n ))
    n = 10
    print("Sum from 1 to",n,"=", sum_1_to_n( n ))


if __name__ == "__main__":
    main()
```

sum m to n



```
def sum_m_to_n( m, n ):
    sum = 0
    count = ...
```

sum m to n



```
def sum_m_to_n( m, n ):
    sum = 0
    count = m

    while count <= n:
        ...

    return sum
```

sum m to n



```
def sum_m_to_n( m, n ):
    sum = 0
    count = m

    while count <= n:
        sum = sum + count
        count = count + 1
    return sum
```

Loop : Fixed number



```
def draw_stairs(x,y,s):


        _
      _|
    _|
  _|

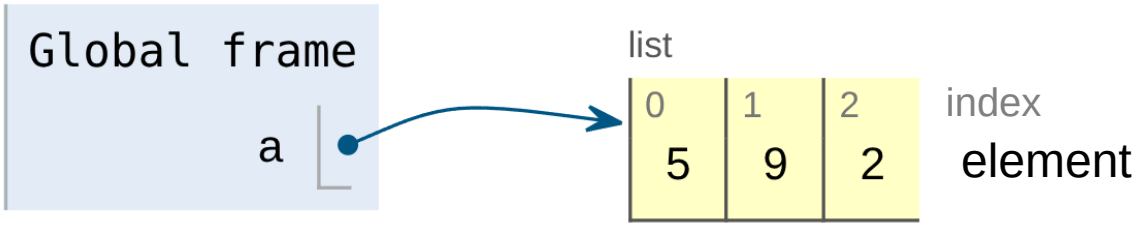

# text-mode or graphic-mode
```

. . .

Array of variables



```
a = [ 5, 9, 2 ]
```




```
# Python use "list" (similar but not the same as array)

# from this example: size or length is 3
# index start from 0 (up to 2)
```

Print elements of Array



```
a = [ 5, 9, 2 ]
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    print( a[i] )     # print element at i
    i = i + 1
```

Sum elements of Array



```
a = [ 5, 9, 2 ]
sum = 0
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    sum = sum + a[i]  # element at index i
    i = i + 1
```




```
# draw flowchart
```

Average?



```
a = [ 5, 9, 2 ]
sum = 0
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    sum = sum + a[i]  # element at index i
    i = i + 1

print( sum/len(a) )
```



Maximum value?



```
a = [ 5, 9, 2 ]
max = a[0]
i = 0  # index
while ( i < len(a)):
    ...
        ...
    i = i + 1
```



Maximum value?



```
a = [ 5, 9, 2 ]
max = a[0]
i = 0  # index
while ( i < len(a)):
    if max < a[i]:
        max = a[i]
    i = i + 1
```



Maximum value?



```
a = [ 5, 9, 2 ]
max = a[0]
i = 1   # index can start from 1?
while ( i < len(a)):
    if max < a[i]:
        max = a[i]
    i = i + 1
```



is_sorted?



```
a = [ 5, 9, 2 ]


i = 0
while ( i < ... ):
    if a[i] < a[i+1]:
        ...
    i = i + 1
```

is_sorted?



```
a = [ 5, 9, 2 ]


i = 0
while ( i < (len(a)-1) ):
    if a[i] < a[i+1]:
        ...
    i = i + 1
```

is_sorted?



```
a = [ 5, 9, 2 ]


i = 0
while ( i < (len(a)-1) ):
    if a[i] > a[i+1]: # ?
        ...
    i = i + 1
```

Algorithms



- find max
- check array is sorted
- sort array?
- calculate factorial

. . .



Parallel Arrays



```
x = [ 1, 4, 2, 9, 3 ]
y = [ 5, 3, 1, 7, 2 ]

# what could they be?
```

Parallel Arrays



```
def draw():
    i = 0
    while ( i < len(x)):
        draw_circle( x[i], y[i], 20)
        y[i] = y[i] + 2

        i = i + 1
```

Start from empty list



```
n = 10
x = [ ]
y = [ ]
```

Add (append) element



```
def setup():
    size(1000,1000)
    i = 0
    while ( i < n ):
        x.append( random(100,900) )
        y.append( random(100,500) )
        i = i + 1
```

Add speed



```
def setup():
    size(1000,1000)
    i = 0
    while ( i < n ):
        x.append( random(100,900) )
        y.append( random(100,500) )
        s.append( random(1,5) )
        i = i + 1
```

Add speed



```
def draw():
    background(255)
    i = 0
    while ( i < n ):
        draw_circle( x[i], y[i], 20)
        y[i] = y[i] + s[i]
        i = i + 1
```

more Parallel Arrays



```
id = [ 001, 007, 199 ]
health = [100, 70, 80]
score = [50, 120, 90]
```

more Parallel Arrays



```
product = ["Apple", "Banana", "Orange"]
price = [1.20, 0.80, 1.50]
stock = [30, 50, 20]
```

Question?



Review Questions



...

String



```
s = "Hello"  # string of characters
```

String



```
s = "Hello"  # string of characters


print(s)     # display string in text mode
```

String



```
s = "Hello"  # similar to array of characters
i = 0
while(i < len(s)):  # can use the same loop?
    print( ... )
    i = i + 1
```

String



```
s = "Hello"  # similar to array of characters
i = 0
while(i < len(s)):
    print( s[i] )
    i = i + 1
```



String



```
s = "Hello"  # similar to array of characters
i = 0
while(i < len(s)):
    print( s[i] )
    i = i + 1
```




```
# In Python, a character is a string of length one

# but not in Java or C
```

ASCII code



```
s = "Hello"  # similar to array of characters
i = 0
while(i < len(s)):
    print( ord(s[i]) )
    i = i + 1

# In Python, a character is a string of length one

# but not in Java or C
```

```
72
101
108
108
111
```

ASCII code



```
print( ord("a") )  # Output: 97
print( ord('a') )  # same in Python

# single and double quotes give the same string

# but in many languages, 'a' is a character (not string)
```

ASCII code



```
print( ord('a') )  # Output: 97

print( ord('A') )  # Output: 65


# see https://en.wikipedia.org/wiki/ASCII
```

ASCII code



```
print( ord('a') )  # Output: 97

print( ord('A') )  # Output: 65

print( ord('0') )  # Output: 48

print( ord(0) )    # TypeError: ord() expected string of length 1, but int found
```

ASCII code



```
print( chr(97) )    # Output a

# reverse of ord()
```

ASCII code



```
print( chr(97) )    # Output a

print( chr(65) )    # Output A
```

Escape Characters



```
print("Hello\nWorld")
```

```
Hello
World
```

```
print("Python\tProgramming")
```

```
Python  Programming
```

```
print("He said \"Hello\"")
```

```
He said "Hello"
```

String Concatenation (+)



```
s = "Hello"
t = "Joe"
print( s + t )
```

String Concatenation (+)



```
s = "Hello"
t = ""      # t is an empty string
i = 0
while(i < len(s)):   # loop
    ...
    i = i + 1
```

String Concatenation (+)



```
s = "Hello"
t = ""
i = 0
while(i < len(s)):
    t = t + str(i)  # add i to t ?
    i = i + 1
```

str() will convert integer to string

String Concatenation (+)



```
s = "Hello"
t = ""
i = 0
while(i < len(s)):
    t = t + str(i)
    i = i + 1
print( t )
```

01234

String Concatenation (+)



```
s = "Hello"
t = ""
i = 0
while(i < len(s)):
    t = t + str(i) + s[i] + "\n"
    i = i + 1
print( t )
```

```
0H
1e
2l
3l
4o
```

String Concatenation (+)



```
s = "Hello"
t = ""
i = 0
while(i < len(s)):
    t = t + str(i) + "\t" + s[i] + "\n"
    i = i + 1
print( t )
```

```
0   H
1   e
2   l
3   l
4   o
```

Display String (graphic-mode)



```
def draw():
    fill(0)          # text color
    textSize(100)    # text size :)
    text("ABC", mouseX, mouseY)
```

...

Array is mutable



```
a = [ 5, 9, 2 ]

a[0] = 7
```

Can change element in array

String is immutable



```
s = "Hello"

s[0] = "J"   # Error

# TypeError: 'str' object does not support item assignment

# Cannot change element in array
```

Create a new string



```
s = "Hello"

s = "J" + s[4] + s[1]

# Assign it to the same variable
```

Create a new string



```
s = "Hello"

s = "J" + s[4] + s[1]

# Assign it to the same variable

# Similar to earlier example
# t = t + str(i) + "\t" + s[i] + "\n"
```

String Method (Function)



```
s = "Hello"

print( s.upper() )

# Call function in a different way
# variable_name.function_name()
```

```
HELLO
```

String Method (Function)



```
s = "Hello"

print( s.lower() )
```

hello

String Method (Function)



```
s = "A"

print( s.isalpha() )
```

True

String Method (Function)



```
s = "3"

print( s.isalpha() )

# boolean function
```

False

String Method (Function)



```
s = "3"

print( s.isdigit() )
```

True

String Method (Function)



```
s = "3"

print( s.isalnum() )


# See
# https://py.processing.org/reference/string
# https://docs.python.org/3/library/stdtypes.html#string-methods
# (not the main focus)
```

True

...

Assessment



Lecture	10%
Lab	10%
A1	10%
A2	10%
Midterm	30%
Final	30%

(maybe adjusted)

Ideas for A1



Animation & Interactive program

- Help to prepare for midterm exam : maths, physics, english

- What you like or interested

Objective: Review programming fundamentals

(variable, function, conditional, loop, array)

Lab 6



010123138

Computer Programming



Check **Attendance..**



Review



Is x positive?



```
if (x > 0):
    print("x is positive")

if (x < 0):
    print("x is negative")

if (x == 0):
    print("x is zero")
```

Nested conditional statement



```
if (x > 0):
    print("x is positive")
else:
    if (x < 0):
        print("x is negative")
    else:
        print("x is zero")
```

while statements



```
n = 5
while (n > 0):
    print(n)
    n = n - 1

# draw flowchart
```

while statements



```
n = 5              # initialise
while (n > 0):  # condition
    print(n)
    n = n - 1   # update

# draw flowchart
```

Infinite Loop



```
n = 5               # initialise
while (n > 0):  # condition
    print(n)
    # n = n - 1   # do not forget to update
```

Sum from 1 to n



```
n = 5
sum = 0
count = 1

while count <= n:
    sum = sum + count
    count = count + 1

print("Sum =", sum)
```

Define a function



```
def sum_1_to_n( n ):
    sum = 0
    count = 1

    while count <= n:
        sum = sum + count
        count = count + 1
    return sum
```

Call the function



```
n = 5
print("Sum from 1 to",n,"=", sum_1_to_n( n ))

n = 10
print("Sum from 1 to",n,"=", sum_1_to_n( n ))
```

define main()



```
# do not need global variables
# try and see on pythontutor.com

def main():
    n = 5
    print("Sum from 1 to",n,"=", sum_1_to_n( n ))
    n = 10
    print("Sum from 1 to",n,"=", sum_1_to_n( n ))


if __name__ == "__main__":
    main()
```

Loop : Fixed number



```
def draw_ladder(x,y,s):
```

Change to vertical lines



```
draw_vertical_lines(int x1,int y1,int x2,int y2):
    int space = abs(x1-x2)/5
    while (x1 < x2):
        line(x1,y1 - 50, x1, y1 + 50)
        x1 = x1 + space
```

Change to vertical lines



```
draw_vertical_lines(int x1,int y1,int x2,int y2):
    int space = abs(x1-x2)/5  # integer division?
    while (x1 < x2):
        line(x1,y1 - 50, x1, y1 + 50)
        x1 = x1 + space
```

Change to vertical lines



```
draw_vertical_lines(int x1,int y1,int x2,int y2):
    int space = abs(x1-x2)/5   # integer division in many programming lanagues
    while (x1 < x2):
        line(x1,y1 - 50, x1, y1 + 50)
        x1 = x1 + space        # if space is zero?
```

Loop : Fixed number



```
def draw_rain_drops(n):
    i = 0;
    while (i < n):          # draw n rain drops
        x = random(100,400)
        y = random(150,350)
        r = random(3,30)
        draw_circle(x,y,r)
        i = i + 1
```

Loop : Until specified sum/volume



```
def draw_rain_drops_until(volume):
    sum = 0
    while (sum < volume):
        ...
        # sum = sum + r; // simplified calculation for volume?
```

Loop : Until specified condition



```
def draw_rain_drops_until_r_below(int value):
    r_flag = True; // use of flag

    while (r_flag)
        int x = (int) random(100,400)
        int y = (int) random(150,350)
        int r = (int) random(3,30)
        draw_circle(x,y,r)
        if ( r < value ):
            r_flag = False
```

Question?



Review Questions



. . .

Sum from 1 to n



```
n = 5
sum = 0
count = 1

while count <= n:
    sum = sum + count
    count = count + 1

print("Sum =", sum)
```

Sum many variables



```
...
sum = 0
sum = sum + x
sum = sum + y
sum = sum + z
...

# 3 variables ok
# 5, 10, ..?
```

Array of variables



```
a = [ 5, 9, 2 ]
...
sum = 0
sum = sum + x
sum = sum + y
sum = sum + z
...
```

Array of variables



```
a = [ 5, 9, 2 ]
```



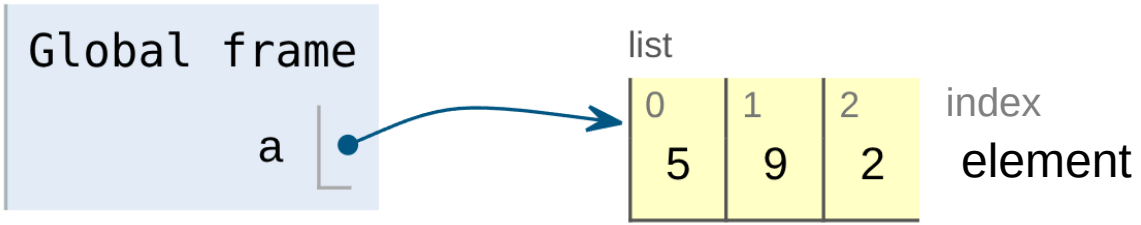

```
# Python use "list" (similar but not the same as array)
```

```
# image from pythontutor.com
```

Array of variables



```
a = [ 5, 9, 2 ]
```



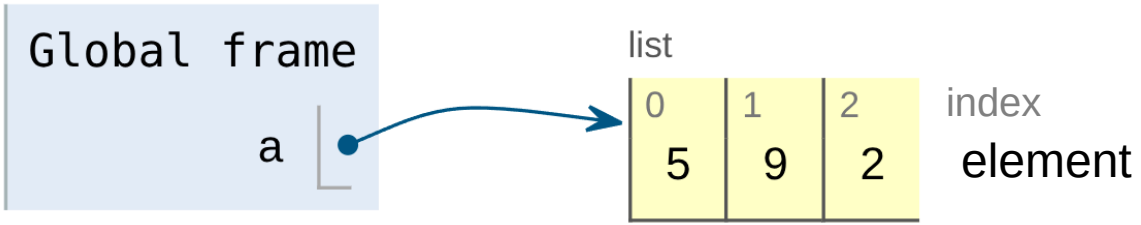

```
# Python use "list" (similar but not the same as array)

# from this example: size or length is 3
# index start from 0 (up to 2)
```

Print elements of Array



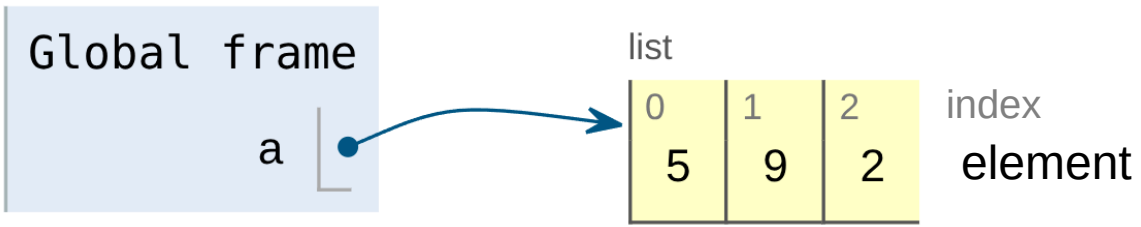
```
a = [ 5, 9, 2 ]
i = 0  # index
```



Print elements of Array



```
a = [ 5, 9, 2 ]
i = 0  # index
```




```
# can i start from ...?
```

Print elements of Array



```
a = [ 5, 9, 2 ]
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    print( a[i] )     # print element at i
    i = i + 1
```

Print elements of Array



```
a = [ 5, 9, 2 ]
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    print( a[i] )     # print element at i
    i = i + 1
```

```
# draw flowchart
```

Show index



```
a = [ 5, 9, 2 ]
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    print("a[", i , "] = ", a[i]);
    i = i + 1
```

```
# draw flowchart
```

...

Sum elements of Array



```
a = [ 5, 9, 2 ]
...
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    ...
    i = i + 1
```

```
# draw flowchart
```

Sum elements of Array



```
a = [ 5, 9, 2 ]
sum = 0   # as before
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    ...
    i = i + 1
```

```
# draw flowchart
```

Sum elements of Array



```
a = [ 5, 9, 2 ]
sum = 0
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    sum = sum + ...
    i = i + 1
```

```
# draw flowchart
```

Sum elements of Array



```
a = [ 5, 9, 2 ]
sum = 0
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    sum = sum + a[i]  # element at index i
    i = i + 1
```




```
# draw flowchart
```

...

Average?



```
a = [ 5, 9, 2 ]
sum = 0
i = 0  # index
while ( i < len(a)):  # len(a) is length of a
    sum = sum + a[i]  # element at index i
    i = i + 1
```




```
# draw flowchart
```

Problem Solving



Find the maximum of 2 values

Problem Solving



Find the maximum of 2 values

if-else?

Problem Solving



Find the maximum of 3 values

Problem Solving



Find the maximum of 3 values

more if-else?

Problem Solving



Find the maximum of 3 values

tmp = max of the first 2 values

result = max of tmp and the third value

Problem Solving



Find the maximum of 4 values

tmp = max of the first 2 values

tmp2 = max of tmp and the third value

result = max of tmp2 and the forth value

Problem Solving



Can only compare 2 values at a time?

Maximum value?



```
a = [ 5, 9, 2 ]
max = ...
```




```
# start from the fist element?
```

Maximum value?



```
a = [ 5, 9, 2 ]
max = a[0]
i = ...


# start from the fist element?
```



Maximum value?



```
a = [ 5, 9, 2 ]
max = a[0]
i = 0  # index
while ( i < len(a)):
    ...
        ...
    i = i + 1
```



Maximum value?



```
a = [ 5, 9, 2 ]
max = a[0]
i = 0  # index
while ( i < len(a)):
    if max < a[i]:
        max = a[i]
    i = i + 1
```



Maximum value?



```
a = [ 5, 9, 2 ]
max = a[0]
i = 1   # index can start from 1?
while ( i < len(a)):
    if max < a[i]:
        max = a[i]
    i = i + 1
```



Buit-in function



```
# for debug
# not in exam : )


print( a )
print( sum(a) )
print( max(a) )
```

...

Parallel Arrays



```
x = [ 1, 4, 2, 9, 3 ]
y = [ 5, 3, 1, 7, 2 ]

# what could they be?
```

Parallel Arrays



```
def draw():
    i = 0
    while ( i < len(x)):

        ...

        i = i + 1
```

Parallel Arrays



```
def draw():
    i = 0
    while ( i < len(x)):
        draw_circle( x[i], y[i], 20)
        ...

        i = i + 1
```

Parallel Arrays



```
def draw():
    i = 0
    while ( i < len(x)):
        draw_circle( x[i], y[i], 20)
        y[i] = y[i] + 2
        ...
        i = i + 1
```

Parallel Arrays



```
def draw():
    i = 0
    while ( i < len(x)):
        draw_circle( x[i], y[i], 20)
        y[i] = y[i] + 2
        x[i] = x[i] + 2 # need it?
        i = i + 1
```

...

Start from empty list



```
n = 10
x = [ ]
y = [ ]
```

Add (append) element



```
def setup():
    size(1000,1000)
    i = 0
    while ( i < n ):
        x.append( random(100,900) )
        y.append( random(100,500) )
        i = i + 1
```

Change n



```
n = ...
x = [ ]
y = [ ]
```

Add speed



```
n = 108
x = [ ]
y = [ ]
s = [ ]
```

Add speed



```
def setup():
    size(1000,1000)
    i = 0
    while ( i < n ):
        x.append( random(100,900) )
        y.append( random(100,500) )
        s.append( random(1,5) )
        i = i + 1
```

Add speed



```
def draw():
    background(255)
    i = 0
    while ( i < n ):
        draw_circle( x[i], y[i], 20)
        y[i] = y[i] + s[i]
        i = i + 1
```

Add?



- Animation

- Interaction

more Parallel Arrays



```
id = [ 001, 007, 199 ]
health = [100, 70, 80]
score = [50, 120, 90]
```

more Parallel Arrays



```
product = ["Apple", "Banana", "Orange"]
price = [1.20, 0.80, 1.50]
stock = [30, 50, 20]
```

...

Assessment



Lecture	10%
Lab	10%
A1	10%
A2	10%
Midterm	30%
Final	30%

(maybe adjusted)

Assessment



Lecture 10% consists of

- After class quizzes (individual)
- Task 1 (group work)
- Task 2 (group work)

(maybe adjusted)

Assessment



Lecture 10% consists of

- Task 1 (group work)
  - find exercises (coding question) and write solution
  - create quizzes (google form)

w1 for group 1, w2 for group 2, ...

Assessment



Lecture	10%
Lab	10%
A1	10%
A2	10%
Midterm	30%
Final	30%

(maybe adjusted)

Ideas for A1



Animation & Interactive program

- Help to prepare for midterm exam : maths, physics, english

- What you like or interested

Objective: Review programming fundamentals

(variable, function, conditional, loop, array)

Lab 5



010123138

Computer Programming



Check **Attendance..**



Review



Simple conditional statement (if)



```
def draw():
    global x
    draw_balloon(x, 200)
    x = x + 10
    if x > 500:
        x = 100
```

Function with return value



```
def draw():
    x = random(100,400) # assign it to a variable
    y = random(150,350) # (store it) , not print/show
    draw_circle(x,y, random(3,30) )

# send it to another function :)
```

User defined Function with return value



```
def cal_distance(x1, y1, x2, y2):
    diffx = abs( x1-x2 )
    diffy = abs( y1-y2 )
    result = sqrt( diffx*diffx + diffy*diffy )
    return result

# Tell the function to return the value of result
# and exit the function
```

Show distance using a line



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)

    line(x,200, x + d, 200)
```

Draw on the same side as the mouse



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)

    if mouseX > x:
        line(x,200, x + d, 200)
    else:
        line(x,200, x - d, 200)
```

Is x positive?



```
if (x > 0):
    print("x is positive")

if (x < 0):
    print("x is negative")

if (x == 0):
    print("x is zero")
```

Nested conditional statement



Similar idea to parenthesis ()

```
x = (a + b)
y = ( (i + j) * ( i - a) / 5 )
```

Nested conditional statement



```
if (x > 0):
    print("x is positive")
else:
    if (x < 0):
        print("x is negative")
    else:
        print("x is zero")
```

Nested conditional statement



```
if (x > 0):
    print("x is positive")
else:
    if (x < 0):
        print("x is negative")
    #else:
        print("x is zero")


# same result?
# draw flowchart?
# try and see on pythontutor.com
```

Nested conditional statement



```
if (x > 0):
    print("x is positive")
else:
    if (x < 0):
        print("x is negative")
    #else:
    print("x is zero")


# same result?
# draw flowchart?
# try and see on pythontutor.com
```

Question?



Review Questions



Assessment



Lecture	10%
Lab	10%
A1	10%
A2	10%
Midterm	30%
Final	30%

(maybe adjusted)

Assessment



Lecture 10% consists of

- After class quizzes (individual)
- Task 1 (group work)
- Task 2 (group work)

(maybe adjusted)

Assessment



Lecture 10% consists of

- Task 1 (group work)
  - find exercises (coding question) and write solution
  - create quizzes (google form)

w1 for group 1, w2 for group 2, ...

Assessment



Lecture	10%
Lab	10%
A1	10%
A2	10%
Midterm	30%
Final	30%

(maybe adjusted)

Ideas for A1



Animation & Interactive program

- Help to prepare for midterm exam : maths, physics, english

- What you like or interested

Objective: Review programming fundamentals

(variable, function, conditional, loop, array)

. . .

Variable



```
n = 5
print(n)

n = n - 1
print(n)
```

Variable



```
n = 5
print(n)

n = n - 1
print(n)

# draw flowchart
```

Variable



```
n = 5
print(n)

n = n - 1
print(n)

n = n - 1
print(n)

n = n - 1
print(n)

n = n - 1  # แปรเปลี่ยนไป
print(n)   # ผลลัพท์เปลี่ยนไป
```

Conditional



```
n = 5
if (n > 0):
    print(n)
    n = n - 1

# draw flowchart
```

Conditional



```
n = 5
if (n > 0):
    print(n)
    n = n - 1

if (n > 0):
    print(n)
    n = n - 1
```

Conditional



```
n = 5
if (n > 0):
    print(n)
    n = n - 1

if (n > 0):
    print(n)
    n = n - 1

if (n > 0):
    print(n)
    n = n - 1

if (n > 0):
    print(n)
    n = n - 1

if (n > 0):
    print(n)
    n = n - 1
```

Conditional



```
...
if (n > 0):
    print(n)
    n = n - 1

# more ?
if (n > 0):
    print("Yes, the condition is true")
    print(n)
    n = n - 1
else:
    print("Sorry, the condition is false")
```

Conditional



```
...

# more ?
if (n > 0):
    print("Yes, the condition is true")
    print(n)
    n = n - 1
else:
    print("Sorry, the condition is false")

if (n > 0):
    print("Yes, the condition is true")
    print(n)
    n = n - 1
else:
    print("Sorry, the condition is false")
```

Repeated statements



```
n = 5
if (n > 0):
    print(n)
    n = n - 1
...
if (n > 0):
    print(n)
    n = n - 1
```

while statements



```
n = 5
while (n > 0):
    print(n)
    n = n - 1
```

while statements



```
n = 5
while (n > 0):
    print(n)
    n = n - 1

# draw flowchart
```

while statements



```
n = 5               # initialise
while (n > 0):  # condition
    print(n)
    n = n - 1   # update

# draw flowchart
```

Infinite Loop



```
n = 5               # initialise
while (n > 0):  # condition
    print(n)
    # n = n - 1   # do not forget to update
```

...

Sum



sum from 1 to n

1+2+3+...

draw flowchart

```
sum = 0  # start
```

Random (Text-mode)



```
import random
x = random.randint(-5, 5)


print(x)
if x < 0:
    print("x is negative")
else:
    print("x is not negative")
```

Loop until



input/random and sum, until **-1** is entered

draw flowchart

Password



```
# input password (not show it)

import getpass
user_password = getpass.getpass("Enter your password: ")
if user_password == "secret123":
    print("Access granted :)")
else:
    print("Access denied!")
```

Password



```
# add loop to limit number of tries

count = 0    # start
```

House



print house, start with floor and wall,then roof?

```
   ^
  ^ ^
 ^   ^
|||||
|   |
|   |
|||||
```

print_line()



```
def print_line(n):

    while (n > 0):
        print("*",end="")    # do not add a newline
        n = n - 1
    print()                  # add a newline
```

...

Loop : Fixed number



```
def draw_ladder(x,y,s):
```

Change to vertical lines



```
draw_vertical_lines(int x1,int y1,int x2,int y2):
    int space = abs(x1-x2)/5
    while (x1 < x2):
        ...
```

Change to vertical lines



```
draw_vertical_lines(int x1,int y1,int x2,int y2):
    int space = abs(x1-x2)/5
    while (x1 < x2):
        line(x1,y1 - 50, x1, y1 + 50)
        x1 = x1 + space
```

Loop : Fixed number



```
def draw_rain_drops(n):
    i = 0;
    while (i < n):                  # draw n rain drops
        x = random(100,400)
        y = random(150,350)
        r = random(3,30)
        draw_circle(x,y,r)
        i = i + 1
```

Loop : Until specified sum/volume



```
def draw_rain_drops_until(volume):
    sum = 0
    while (sum < volume):
        ...
        # sum = sum + r; // simplified calculation for volume?
```

Loop : Until specified condition



```
def draw_rain_drops_until_r_below(int value):
    r_flag = True;   // use of flag

    while (r_flag){

        int x = (int) random(100,400)
        int y = (int) random(150,350)
        int r = (int) random(3,30)

        draw_circle(x,y,r)
        if ( r < value ):
            r_flag = False
```

...

Lab 4



010123102

Programming Fundamentals



Check **Attendance..**



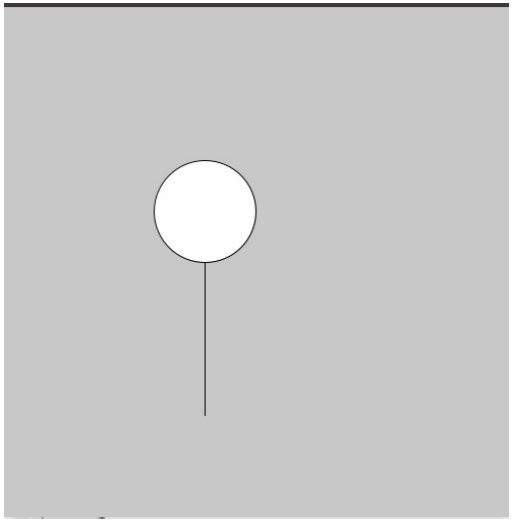
Review



Simple Output



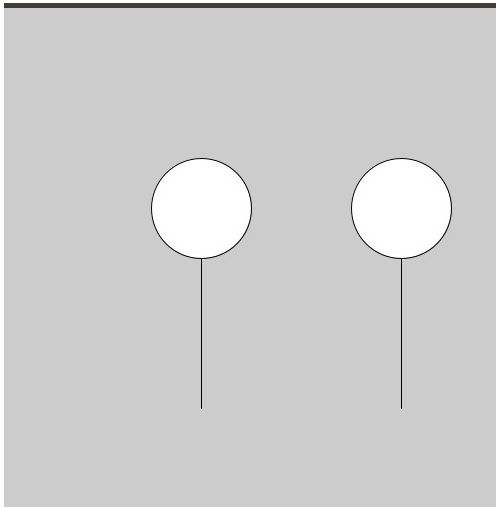
```
size(500,500)
# draw balloon
ellipse(200,200,100,100)
line(200,250,200,400)
```



Simple Output



```
size(500,500)
# draw balloon
ellipse(200,200,100,100)
line(200,250,200,400)
```



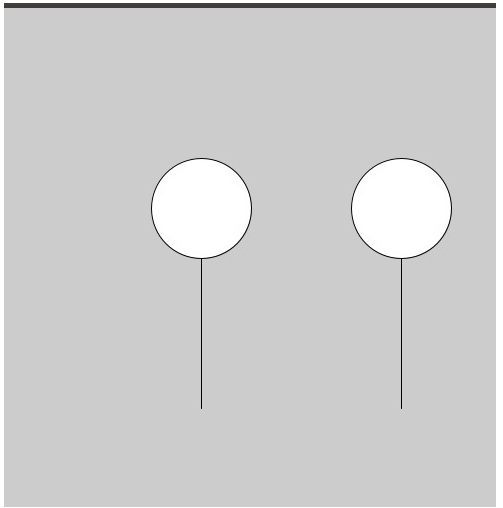
```
# draw another balloon
ellipse(400,200,100,100) # change value of parameters/arguments
line(400,250,400,400)
```

Use Variable



```
size(500,500)
x = 200
y = 200
# draw balloon
ellipse(x, y, 100, 100)
line(x, y+50, x, y+200)

x = x + 200                    # change value of variable
# draw another balloon
ellipse(x, y, 100, 100)        # same code?
line(x, y+50, x, y+200)
```



Define draw_balloon()



```
def draw_balloon( ... ):
    ellipse(x, y, 100, 100) # same code inside the function
    line(x, y+50, x, y+200)
```

Use Parameters



```
def draw_balloon( x, y ):
    ellipse(x, y, 100, 100) # same code inside the function
    line(x, y+50, x, y+200)
```

When called, it will send arguments



```
def draw_balloon(x, y):
    ellipse(...)
    line(...)


draw_balloon(200, 200)
# the values will be copied
```

Copied to called function



```
def draw_balloon(x, y):
    x = 200 # similar result, as adding these two lines of code
    y = 200
    ellipse(...)
    line(...)

draw_balloon(200, 200)
# the values will be copied
```

Copied to called function



```
def draw_balloon(x, y):
    x = 200 # similar result , but do not write the code
    y = 200
    ellipse(...)
    line(...)

draw_balloon(200, 200)
# the values will be copied
```

Parameters are local variables



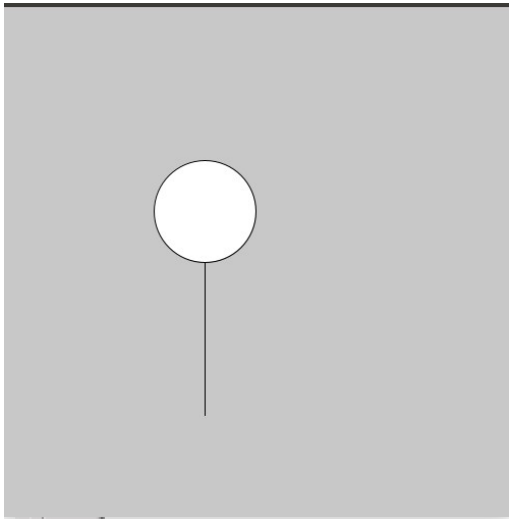
```
def draw_balloon( x, y ):
    ellipse(x, y, 100, 100)
    line(x, y+50, x, y+200)
```

Only 2 lines of code in draw_balloon()

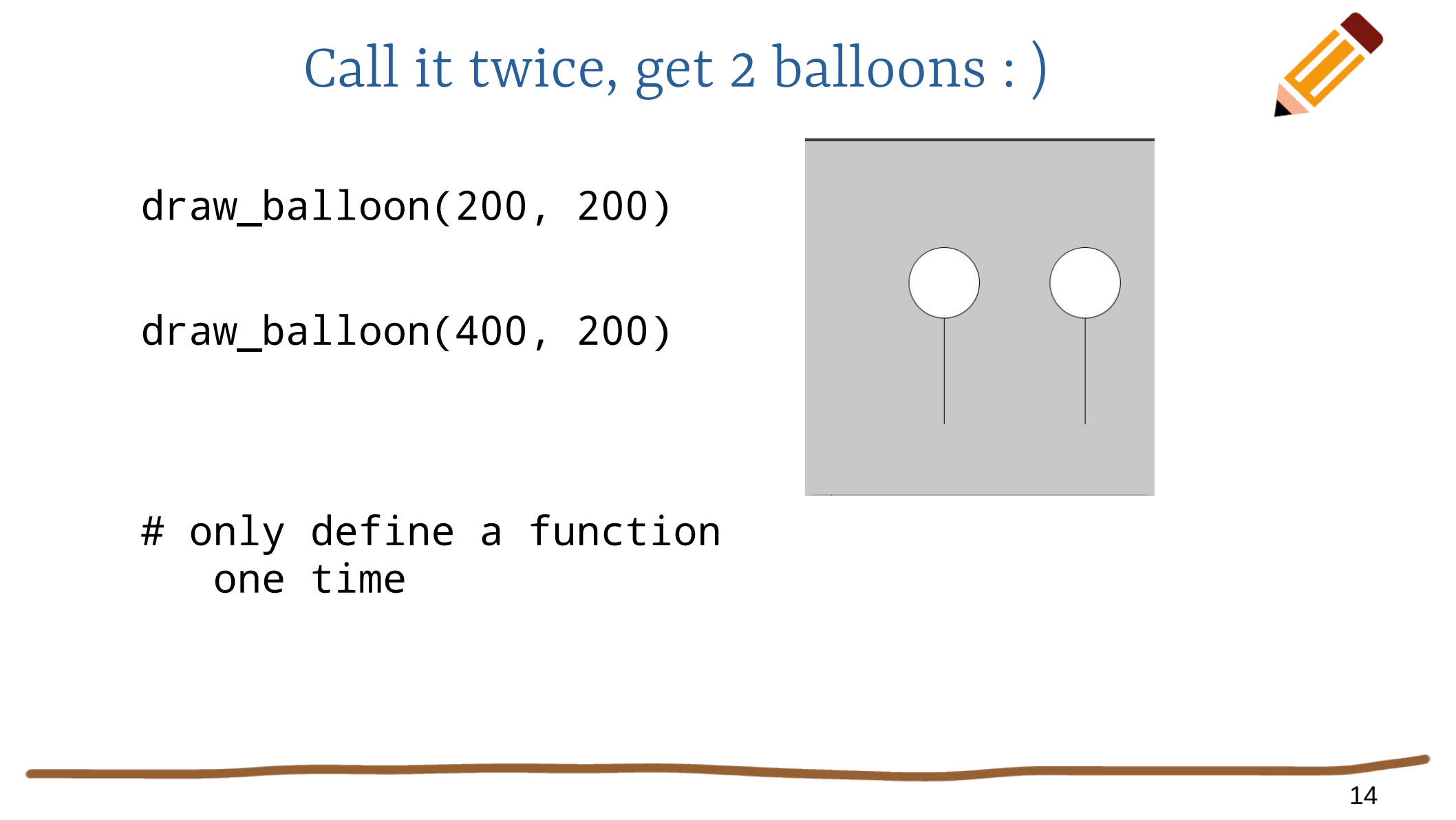


```
def draw_balloon(x, y):
    ellipse(x, y, 100, 100)
    line(x, y+50, x, y+200)

draw_balloon(200, 200)
```



Call it twice, get 2 balloons :)



```
draw_balloon(200, 200)

draw_balloon(400, 200)



# only define a function
    one time
```

Use variables



```
# ...
draw_balloon(x, y)

draw_balloon(x, y)
```

Use variables



```
x = 200
y = 200
draw_balloon(x, y)

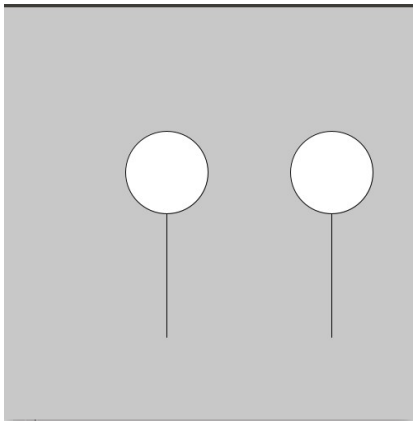
draw_balloon(x, y)    # what will be shown?
```

Use variables



```
x = 200
y = 200
draw_balloon(x, y)

x = x + 200                # must update value of x
draw_balloon(x, y)          # (to draw 2 balloons)
```



Animation with a balloon



using

- setup()
- draw()

- global variable

Global variable



```
def setup():
    size(500,500)

def draw():
    global x                    # where is it?
    draw_balloon(x, 200)
    x = x + 10
```

Global variable



```
x = 200                          # not inside a function
def setup():
    size(500,500)

def draw():
    global x
    draw_balloon( x , 200)
    x = x + 10
```

Try with local variable



```
x = 200                          # 2 variables named x
def setup():
    size(500,500)

def draw():
    x = 200      # local variable, inside a function
    draw_balloon( x , 200)
    x = x + 10
```

Why does not work?



```
x = 200                          # this x is global
def setup():
    size(500,500)

def draw():
    x = 200                      # this x is local
    draw_balloon( x , 200)
    x = x + 10
```

Why does not work?



```
x = 200                          # this x is global
def setup():
    size(500,500)
                                 # They are not the same x
def draw():
    x = 200                      # this x is local
    draw_balloon( x , 200)
    x = x + 10
```

Why does not work?



```
x = 200
def setup():
    size(500,500)

def draw():
    x = 200                          # no animation?
    draw_balloon( x , 200)           # same value of x
    x = x + 10                       # every time draw() is called
```

Why does not work?



```
x = 200
def setup():
    size(500,500)

def draw():
    x = 200
    draw_balloon(200, 200)    # similar result?
    x = x + 10
```

Simple conditional statement (if)



```
def draw():
    global x
    draw_balloon(x, 200)
    x = x + 10
    if x > 500:
        x = 100
```

Global variables



```
x = 100 # user defined

def draw():
    global x
    draw_balloon(x, 200)
    ...
```

Global variables



```
def draw():
    global x
    draw_balloon(x, mouseY)    # built-in
    x = x + 10
    if x > width:              # built-in
        x = 100
```

Question?



Review Questions



Another user defined Function



```
draw_balloon()
```

User defined Function



```
def draw_circle(x, y, r):  # using radius?
    ...
```

User defined Function



```
def draw_circle(x, y, r):
    ellipse(x,y, r*2, r*2)

    # call built-in function, ellipse()
    # with appropriate parameters

    # effectively drawing a circle with radius r
```

Use it in our function



```
def draw_balloon(x, y):
    ellipse(x, y, 100, 100)     # replace it?
    line(x, y+50, x, y+200)
```

Use it in our function



```
def draw_balloon(x, y):
    draw_circle(x,y, 50)          # easier to understand?
    line(x, y+50, x, y+200)
```

Use it in another function



```
def draw_balloon(x, y):
    # ellipse(x, y, 100, 100)     # replace it
    draw_circle(x,y, 50)          # easier to understand?
    line(x, y+50, x, y+200)
```

To try and use it in other functions



```
draw_bike()
draw_donut()
draw_target() # bullseye

draw_snowman()
draw_ladybug()
draw_smiley_face()
...
```

Built-in Function



```
# Python
```

- `print()`

```
# Python Mode for Processing
```

- `line()`
- `ellipse()`
- `background()`

Built-in Function



```
# Python
```

- `print("Hello")`

```
# Python Mode for Processing
```

- `line(5, 5, 5, 5)`
- `ellipse(9, 9, 9 ,9)`
- `background(11)`

```
# Output according to parameters
```

random()



```
# Python Mode for Processing
```

- `random()`

```
# Generates a random number

# and return a value according to parameters
```

random( )



```
x = random(100,400)
```

```
# The function will return a value between 100 and 400
# and the return value will be assigned to x
```

random( )



```
def draw():
    x = random(100,400) # random value of x
    y = random(150,350) # random value of y
```

random( )



```
def draw():
    x = random(100,400) # random value of x
    y = random(150,350) # random value of y
    draw_circle(x,y,30)
```

Raining?



```
def draw():
    x = random(100,400)
    y = random(150,350)
    draw_circle(x,y,30)


# Very fast, set frame rate by using frameRate(5)
# in setup()
```

Function with return value



```
def draw():
    x = random(100,400) # assign it to a variable
    y = random(150,350) # (store it) , not print/show
    draw_circle(x,y, 30 )
```

Function with return value



```
def draw():
    x = random(100,400) # assign it to a variable
    y = random(150,350) # (store it) , not print/show
    draw_circle(x,y, random(3,30) )

# send it to another function :)
```

...

User defined Function with return value



```
def cal_distance(x1, y1, x2, y2):

# To calculate the distance between 2 points
# and return the result

    # x1: The x-coordinate of the first point.
    # y1: The y-coordinate of the first point.
    # x2: The x-coordinate of the second point.
    # y2: The y-coordinate of the second point.
```

User defined Function with return value



```
def cal_distance(x1, y1, x2, y2):
    diffx = abs( x1-x2 )

# calculates the absolute difference
#   between the x-coordinates of the two points
```

User defined Function with return value



```
def cal_distance(x1, y1, x2, y2):
    diffx = abs( x1-x2 )

# calculates the absolute difference
#   between the x-coordinates of the two points

# abs() is a built-in function that returns
# the absolute value of a number (makes it positive)
```

User defined Function with return value



```
def cal_distance(x1, y1, x2, y2):
    diffx = abs( x1-x2 )
    diffy = abs( y1-y2 )

# Similarly, calculates the absolute difference
# between the y-coordinates
```

User defined Function with return value



```
def cal_distance(x1, y1, x2, y2):
    diffx = abs( x1-x2 )
    diffy = abs( y1-y2 )
    result = sqrt( diffx*diffx + diffy*diffy )

# Calculates the result
# using Pythagorean theorem : )
```

User defined Function with return value



```
def cal_distance(x1, y1, x2, y2):
    diffx = abs( x1-x2 )
    diffy = abs( y1-y2 )
    result = sqrt( diffx*diffx + diffy*diffy )
    return result

# Tell the function to return the value of result
# and exit the function
```

Cal distance to mouse position



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)
```

Show distance using a line



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)

    line(x,200, x + d, 200)
```

...

Draw on the same side as the mouse



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)

    ?
```

Draw on the same side as the mouse



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)

    if mouseX > x:
        line(x,200, x + d, 200)
    ?
```

Draw on the same side as the mouse



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)

    if mouseX > x:
        line(x,200, x + d, 200)
    else:
        line(x,200, x - d, 200)
```

Never draw on both sides?



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)

    if mouseX > x:
        line(x,200, x + d, 200)
    else:
        line(x,200, x - d, 200)
```

Draw Flowchart?



```
def draw():
    ...
    draw_balloon(x, 200, 50)
    d = cal_distance(x, 200, mouseX, mouseY)

    if mouseX > x:
        line(x,200, x + d, 200)
    else:
        line(x,200, x - d, 200)
```

...

Is x positive?



Is x negative?



Is x zero?



Draw Flowchart?



Different Flowchart but the same result?



...

Nested conditional statement



Nested conditional statement



Similar idea to parenthesis ()

Nested conditional statement



Similar idea to parenthesis ()

```
x = (a + b)
y = ( (i + j) * ( i - a) / 5 )
```

...

To Try (and fix errors :)



```
show_info()   # showInfo() just explains what this program does

width = get_width()    # get width from user
length = get_length()  # get length from user

area = cal_area(width, length)  # calculate result

show_area(area)         # show result
show_area_in_SqWa(area)
```

Lab 3



...

Ideas for A1



Animation & Interactive program

- Help to prepare for midterm exam : maths, physics, english

Ideas for A1



Animation & Interactive program

- Help to prepare for midterm exam : maths, physics, english

- What you like or interested

Objective: Review programming fundamentals

(variable, function, conditional, loop, array)

...

010123102

Programming Fundamentals



Check **Attendance..**



Review



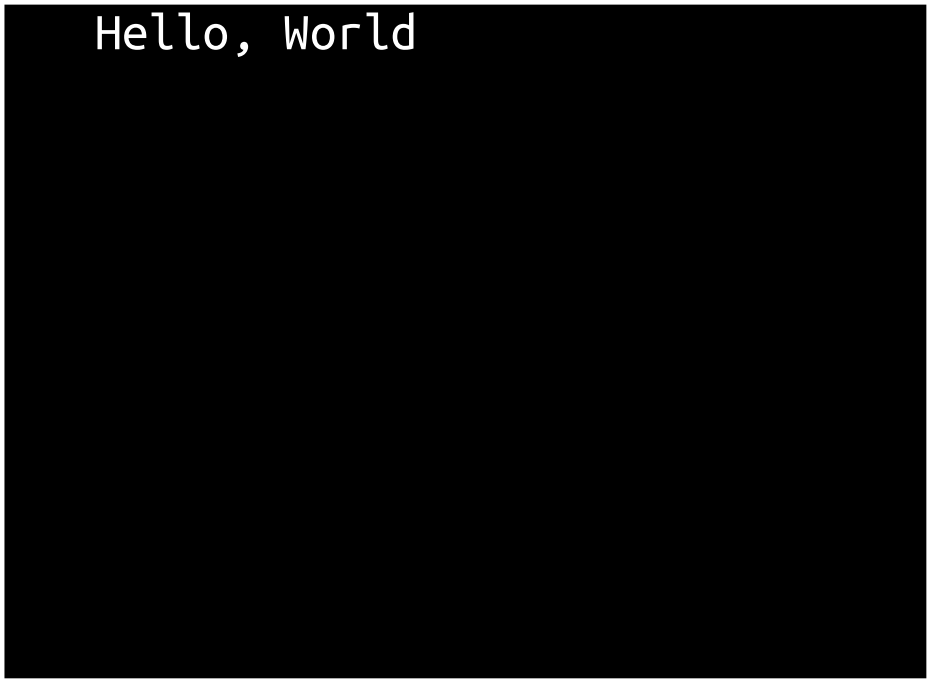
Start from **Output** ?



Input → Process → Output

Start from Output

```
print("Hello, World")
```



- To check that installed program works correctly
- also verify that it can follow instruction

  and give feedback **`Hello, World`**

Simple Output



```
size(500,500)
line(250,250,0,0)
line(250,250,500,0)
line(250,250,250,500)
```



Different code, same **Output**



```
size(500,500)
line(250,250,250,500) # |
line(250,250,500,0) # /
line(250,250,0,0) # \
```

```
size(500,500)
line(250,250,500,0) # /
line(250,250,250,500) # |
line(250,250,0,0) # \
```

Simple Calculation



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)
```

Simple Calculation



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)
```



Simple Calculation



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)

# to draw letter N
```



Simple Calculation



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)

# to draw letter H
```



Use Variables



```
size(500,500)
x = 200 # upper left x
y = 100 # upper left y
line(x, y, x, y+200)
line(x+100, y, x+100, y+200)

# to draw letter H
```



Use Variables



```
size(500,500)
x = 200 # upper left x
y = 100 # upper left y
line(x, y, x, y+200)
line(x+100, y, x+100, y+200)
```

```
middle = y + 100
line(x,middle,x+100,middle)
```



Built-in Function



```
# Python
```

- `print()`

```
# Python Mode for Processing
```

- `line()`
- `ellipse()`

Question?



Lab1 submission



check that you submitted
the correct URL

:)

Review Questions



...

draw Ball



```
ellipse(200,200,100,100)
```

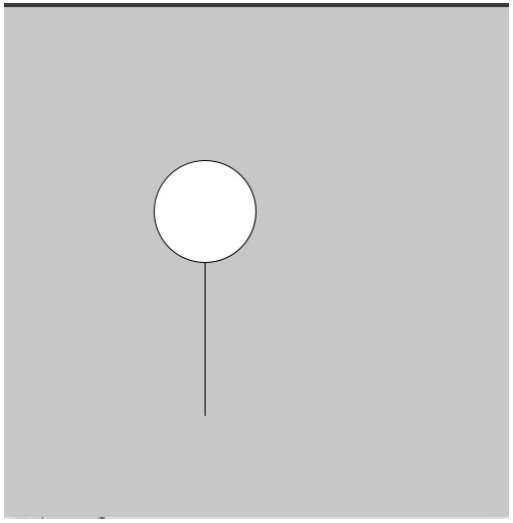
```
# Python Mode for Processing
```

- `line()`
- `ellipse()`

draw Balloon



```
ellipse(200,200,100,100)
line(200,250,200,400)
```



Define draw_balloon()



```
def draw_balloon():
    ellipse(200,200,100,100)
    line(200,250,200,400)
```

Define draw_balloon()



```
def draw_balloon():                    # user-defined function
    ellipse(200,200,100,100)
    line(200,250,200,400)
```

Define draw_balloon()



```
def draw_balloon():
    ellipse(200,200,100,100)
    line(200,250,200,400)
```

```
# user-defined function
# not built-in (built-eng :)
```

Run the code



```
def draw_balloon():
    ellipse(200,200,100,100)
    line(200,250,200,400)
```

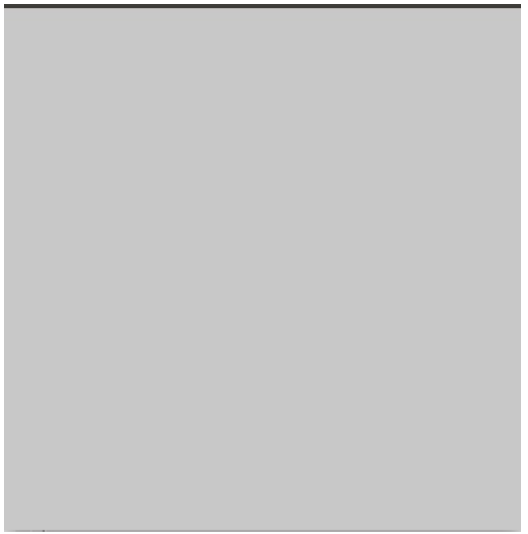
```
# will draw a ballon?
```

Run the code



```
def draw_balloon():
    ellipse(200,200,100,100)
    line(200,250,200,400)

# nothing?
```



Must call the function



```
def draw_balloon():
    ellipse(200,200,100,100)
    line(200,250,200,400)

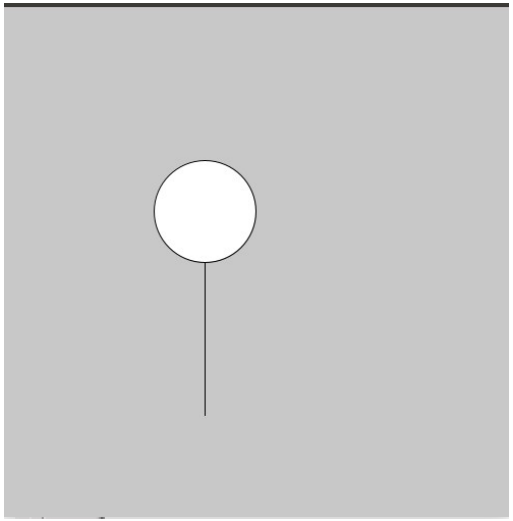
# how?
```

Call its name



```
def draw_balloon():
    ellipse(200,200,100,100)
    line(200,250,200,400)

draw_balloon()
# its name followed by ()
```

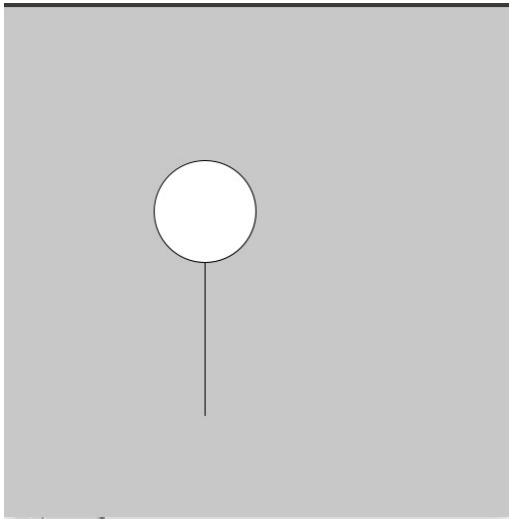


Add parameter



```
def draw_balloon(x, y):
    ellipse(...)
    line(...)

draw_balloon(200, 200)
```

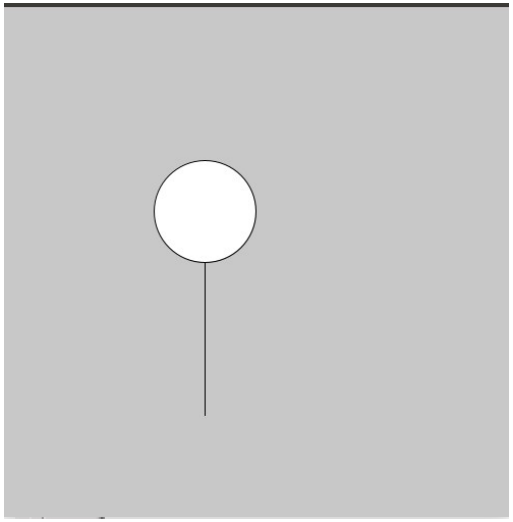


When called, it will send arguments



```
def draw_balloon(x, y):
    ellipse(...)
    line(...)

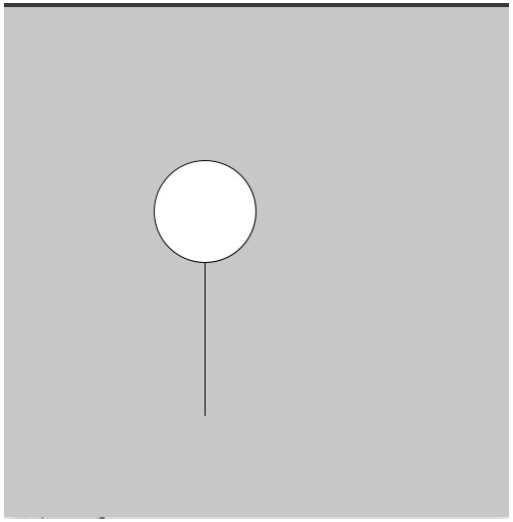

draw_balloon(200, 200)
# the values will be copied
```



Back to called function



```
def draw_balloon(x, y):
    x = 200 # similar result
    y = 200
    ellipse(...)
    line(...)
```

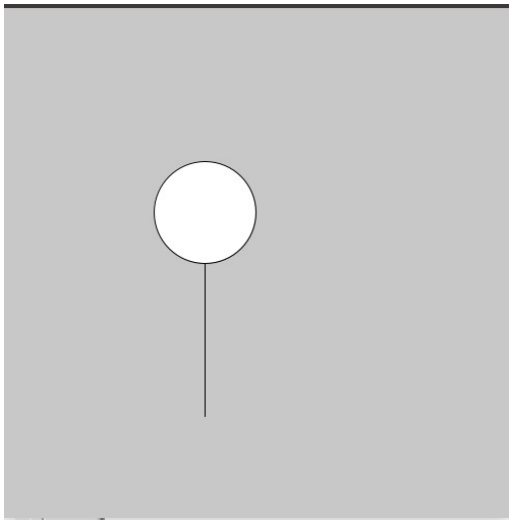


```
draw_balloon(200, 200)
# the values will be copied
```

Back to called function



```
def draw_balloon(x, y):
    x = 200 # similar result
    y = 200
    ellipse(...) # ?
    line(...)
```

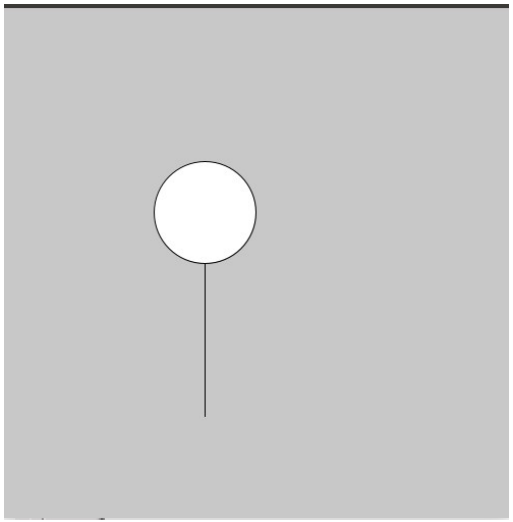


```
draw_balloon(200, 200)
# the values will be copied
```

Back to called function



```
def draw_balloon(x, y):
    x = 200 # similar result
    y = 200
    ellipse(x, y, 100, 100)
     # use the sent argument
    line(...)
```

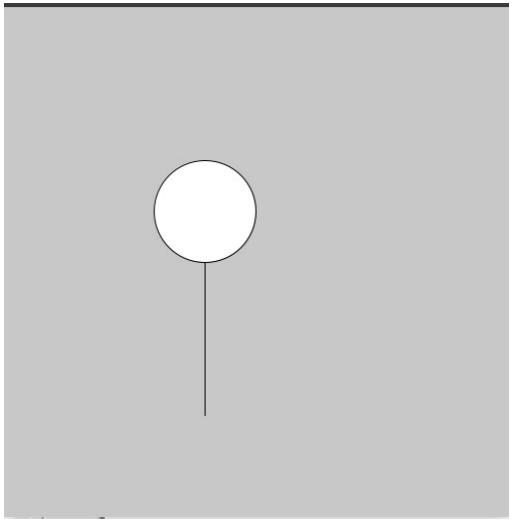


```
draw_balloon(200, 200)
# the values will be copied
```

Back to called function



```
def draw_balloon(x, y):
    x = 200 # similar result
    y = 200
    ellipse(x, y, 100, 100)
    line(x, y+50, x, y+200)
    # here also
```



```
draw_balloon(200, 200)
# the values will be copied
```

Call it twice



```
draw_balloon(200, 200)      # What will be shown?

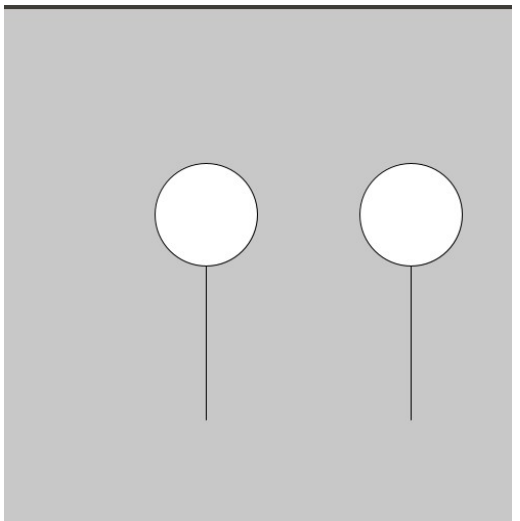
draw_balloon(400, 200)
```

Call it twice, get 2 balloons :)



```
draw_balloon(200, 200)

draw_balloon(400, 200)
```



Call it twice, get 2 balloons :)

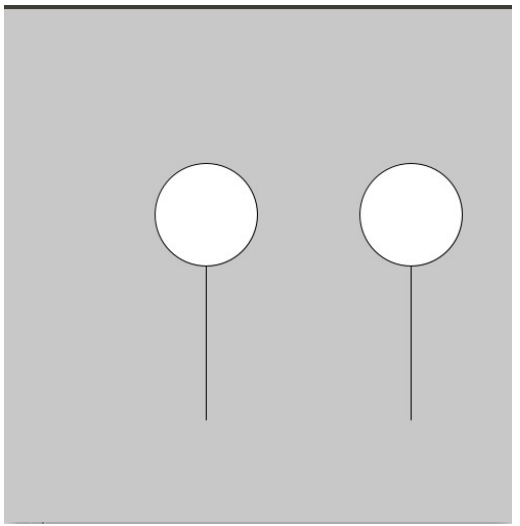


```
draw_balloon(200, 200)

draw_balloon(400, 200)



# only define a function
    one time
```



Animation with a balloon?



Special functions in Processing



```
def setup():   # will be called 1 time
    size(500,500)
```

Special functions in Processing



```
def setup():   # will be called 1 time
    size(500,500)

def draw():    # will be called repeatedly
    # can we proof that?
```

Special functions in Processing



```
def setup():   # will be called 1 time
    size(500,500)


def draw():    # will be called repeatedly
    draw_ballon(200,200)
# does not work
```

Special functions in Processing



```
def setup():   # will be called 1 time
    size(500,500)

def draw():    # will be called repeatedly
    global x
    draw_balloon(x, 200)
    x = x + 10
# still does not work?
```

Simple conditional statement (if)



```
def draw():    # will be called repeatedly
    global x
    draw_balloon(x, 200)
    x = x + 10
    if x > 500:
        x = 100
```

Global variables



```
# user-defined
x = 100
def draw():    # will be called repeatedly
    global x
    draw_balloon(x, 200)
    x = x + 10
    if x > 500:
        x = 100
```

Global variables



```
x =100
def draw():    # will be called repeatedly
    global x
    draw_balloon(x, 200)
    x = x + 10
    if x > width: # built-in
        x = 100
```

Global variables



```
x =100
def draw():    # will be called repeatedly
    global x
    draw_balloon(x, mouseY) # built-in
    x = x + 10
    if x > width:
        x = 100
```

...

Text Mode



```
def show_info():
    print("This program will calculate area of a land")
    print("given width and length")
    print()
    print("created by <student name>")

show_info()
```

Text Mode



```
def show_area( area ):
    print("The result area is", area, "(in square meter)")

show_area(100)
```

To Try (and fix errors :)



```
show_info()   # showInfo() just explains what this program does

width = get_width()    # get width from user
length = get_length()  # get length from user

area = cal_area(width, length)  # calculate result

show_area(area)         # show result
show_area_in_SqWa(area)
```

Lab2



010123102

Programming Fundamentals



Hello and Welcome to KMUTNB



Welcome to Computer Engineering



What is "**Engineering**"?

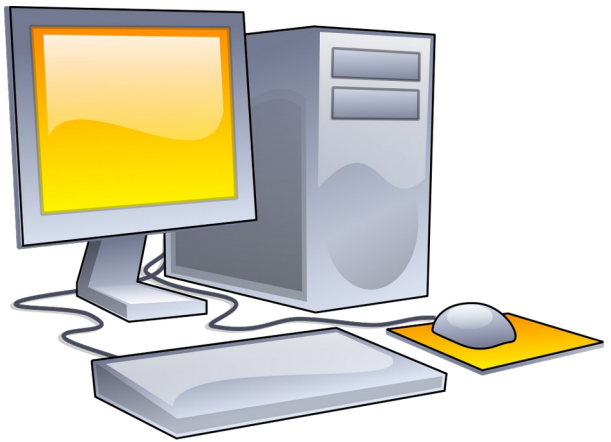


What is "**Engineering**"?

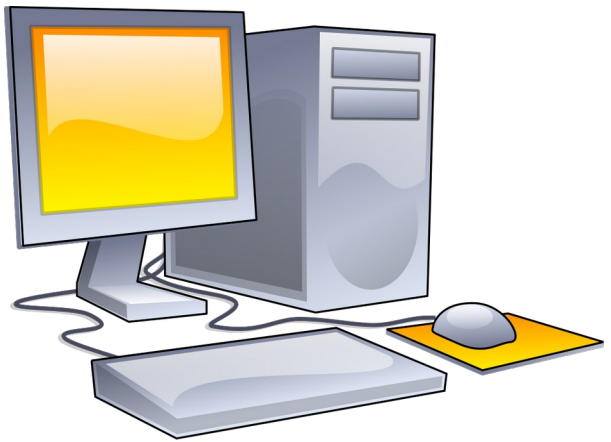




What is "Computer" ?



What is "Computer" ?





?

Check Attendance..



- Need a volunteer

  (youtuber?)

Check Attendance..



- To enter your counted number

  in google form

  https://docs.google.com/forms/d/e/1FAIpQLSdDTX0gqlpnfjRQ5j5kFaWiJevYc7QujzeZdoD5ndBdv0RbEA/viewform?usp=header

What is "Programming" ?



What is "Programming" ?



```
print("Hello, World") ?
```

Why "Programming" ?



How "Programming"?



Start from **Output** ?



Input → Process → Output

Start from Output



```
print("Hello, World")
```

Start from Output



```
print("Hello, World")
```

```
Hello, World
```

- To check that installed program works correctly
- also verify that it can follow instruction

  and give feedback **`Hello, World`**

Change the String



```
print("Hello, KMUTNB")
```

```
Hello, ... ?
```

Change the String



```
print("Hello, KMUTNB")
```

```
Hello, KMUTNB
```

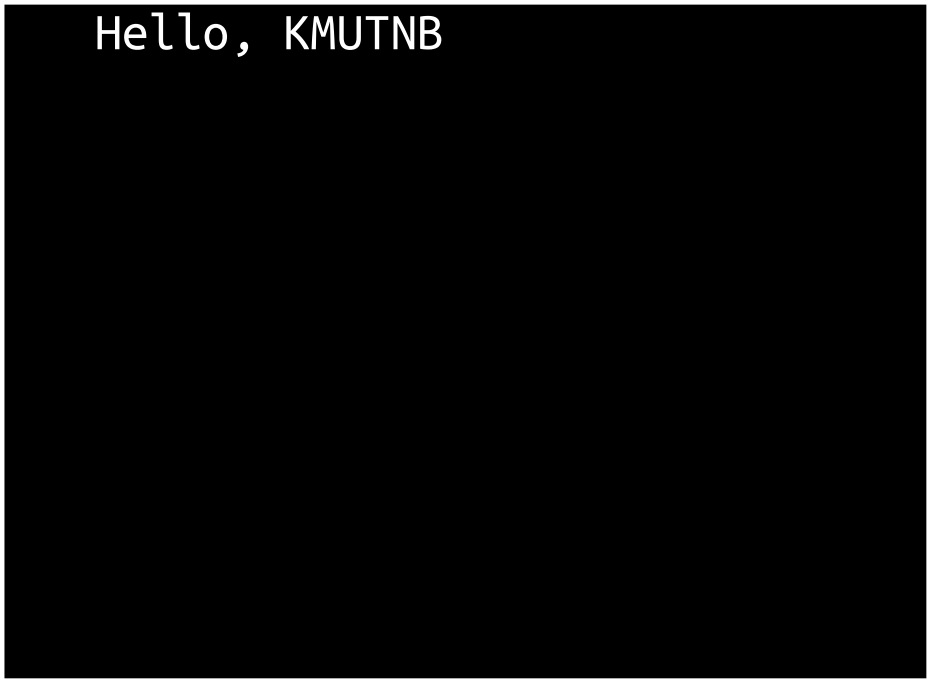
- Put what you want it to say inside " "
  (quotation marks)

Do not like **Text-mode ?**

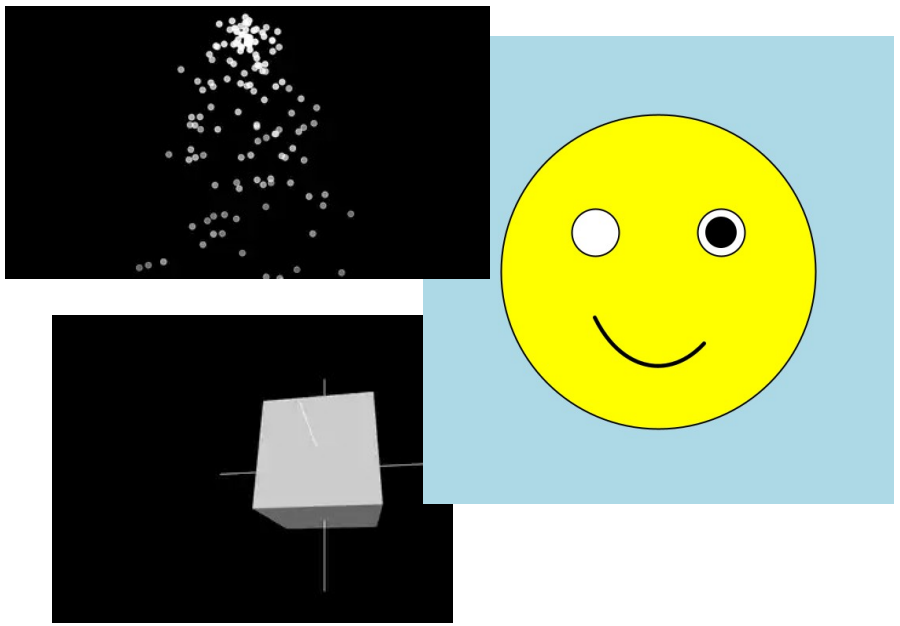


```
print("Hello, KMUTNB")


# display characters only
```



How about Graphic-mode ?



https://processing.org

Processing is a flexible software sketchbook and a language for learning how to code.

To use: Python Mode for Processing

(more in lab1)

Python Mode for Processing

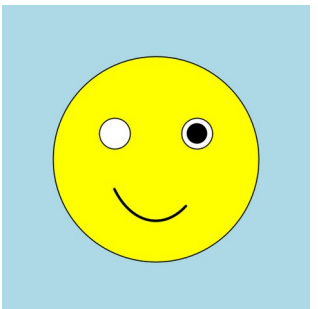


```
    # --- Draw the head (large yellow
circle) ---
    fill(YELLOW)
    stroke(BLACK)
    strokeWeight(2)
    circle(width / 2, height / 2, 200) #
Center the head


    # --- Draw the left eye ---
    # White part of the eye
    fill(WHITE)
    stroke(BLACK)
    strokeWeight(2)
    circle(width / 2 - 80, height / 2 -
50, 30)
```

https://py.processing.org

Write Python code to draw :)



Simple Output

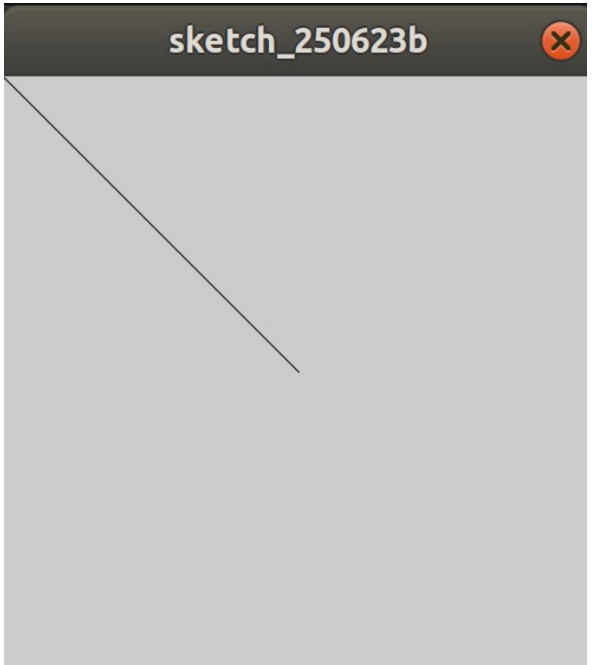


```
size(500,500)
line(0,0,250,250)
```

Simple Output



```
size(500,500)
line(0,0,250,250)
```



Simple Output



```
size(500,500)
line(0,0,250,250)
line(250,250,250,500)
line(250,250,500,0)
```

Simple Output



```
size(500,500)
line(0,0,250,250)
line(250,250,250,500)
line(250,250,500,0)
```



Different code, same **Output**



```
size(500,500)
line(250,250,250,500) # |
line(250,250,500,0) # /
line(0,0,250,250) # \
```

```
size(500,500)
line(250,250,500,0) # /
line(250,250,250,500) # |
line(0,0,250,250) # \
```

Simple Calculation



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)
```

Simple Calculation



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)
```



Simple Calculation



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)

# to draw letter N
```



Simple Calculation



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)

# to draw letter H
```



Use Variables



```
size(500,500)
x = 200 # upper left x
y = 100 # upper left y
line(x, y, x, y+200)
line(x+100, y, x+100, y+200)

# to draw letter H
```



Use Variables



```
size(500,500)
x = 200 # upper left x
y = 100 # upper left y
line(x, y, x, y+200)
line(x+100, y, x+100, y+200)
```

```
middle = y + 100
line(x,middle,x+100,middle)
```



Use Variables



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)

# to draw letter N
```



Use Variables



```
size(500,500)
line(200,100,200,300)
line(300,100,300,300)

# to draw letter U
```



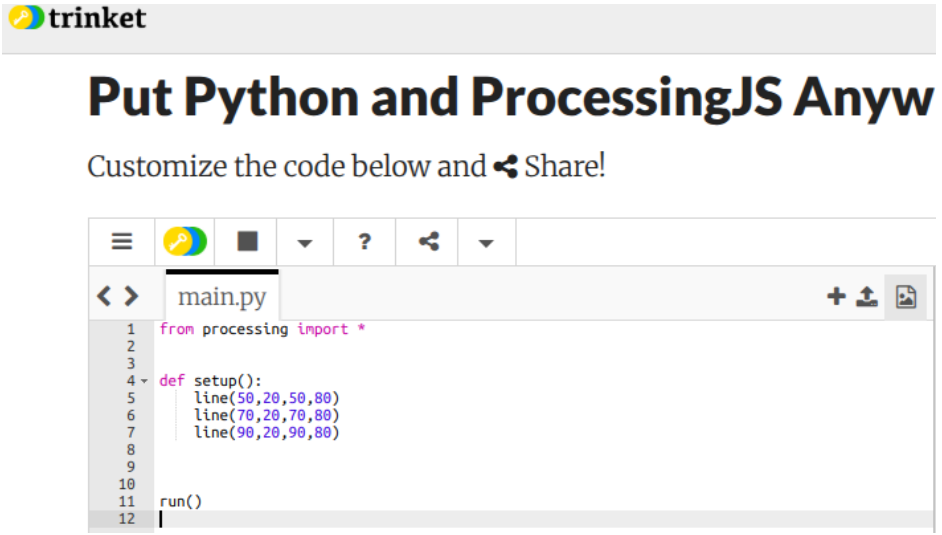
Try it online



```
# Goto https://trinket.io/processing
from processing import *

def setup():
    line(50,20,50,80)
    line(70,20,70,80)
    line(90,20,90,80)

run()
# need to modify the code
```

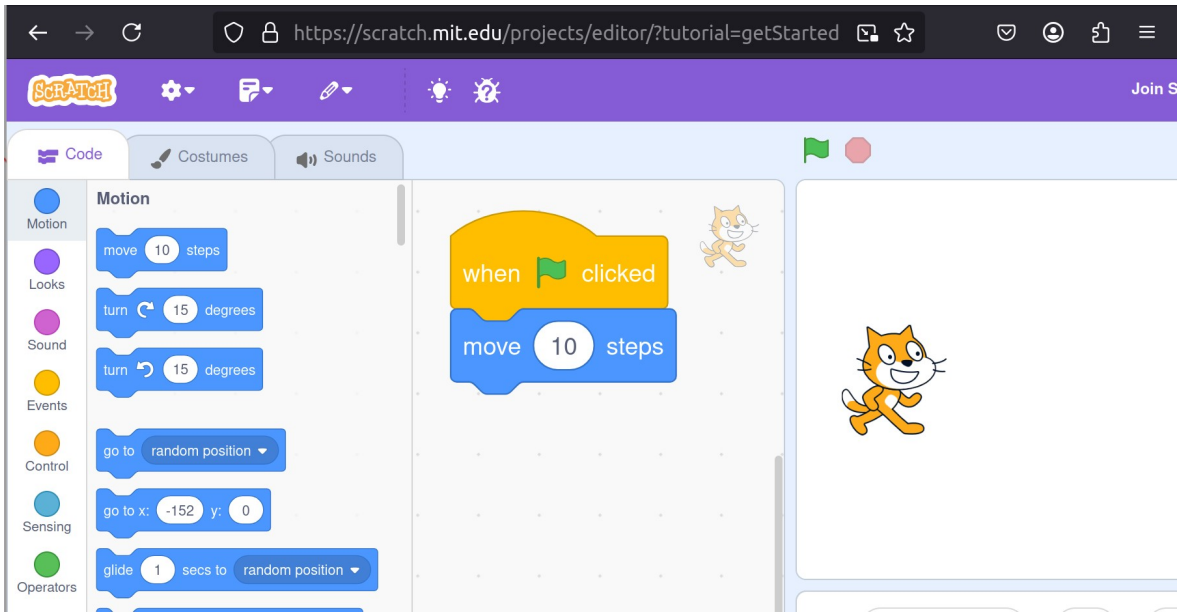


Have you Scratched ?



```
# Goto https://scratch.mit.edu/

# no code
# drag and drop
```



Can you C ?



```
#include <stdio.h>
int main() {

    printf("Hello, World");

}
```

Break



Lab



- 2 hours weekly lab
- Dividing students into 2 groups (odd/even counted number?)
- One group in the morning

  (9-11 am)
- Another group in the afternoon (1-3 pm)

- Lab is on level 3

  (below this lecture room?)

Lab1



- To be posted on google classroom

Quick Quiz :)



?



```
print("KMUT" + "NB")
```

?



```
print("1" + "2")
```

?



```
print(1 + 2)
```

Review Questions



Later this week, check google classroom

class code: fzwp25m3